

UNIVERSIDADE ESTADUAL DE SANTA CRUZ

Larissa de Brito Santos

RELATÓRIO 2: IMPLEMENTAÇÃO PARALELA COM MPI

Ilhéus – BA
2026

Larissa de Brito Santos

RELATÓRIO 2: IMPLEMENTAÇÃO PARALELA COM MPI

Relatório técnico apresentado como requisito parcial para disciplina DEC107 — Processamento Paralelo, referente à etapa 2 do projeto da matéria: Benchmark de Geração do Conjunto de Mandelbrot com MPI.

Professor: Esbel Tomas Valero Orellana

Ilhéus – BA
2026

SUMÁRIO

1 INTRODUÇÃO 4

2 METODOLOGIA 6
2.1 Especificações e Parâmetros do Benchmark 6
2.2 Implementação Sequencial de Referência 7
2.3 Implementação Paralela com MPI 7
2.3.1 Divisão Estática (`mpi_estatico.cpp`) 8
2.3.2 Divisão Dinâmica Tradicional (`mpi_dinamico.cpp`) 8
2.3.3 Divisão Dinâmica com Coordenador Participativo (`mpi_dinamico_participativo.cpp`) 8
2.3.4 Divisão Dinâmica em Meia-Linha (`mpi_dinamico_meia_linha.cpp`) . . . 9
2.4 Casos de Testes e Configurações Avaliadas 9
2.5 Ambiente de Execução e Hardware 9
2.6 Procedimentos de Medição de Tempo e Métricas de Avaliação . . . 11

3 RESULTADOS E DISCUSSÃO 13
3.1 Definição do Tempo Representativo de Referência 13
3.2 Discussão dos Resultados: Caso 1 (Visão Global) 15
3.3 Discussão dos Resultados: Caso 2 (Vale dos Cavalos-Marinhos) . . . 17
3.4 Análise do Desbalanceamento de Carga 19
3.5 Escalabilidade e Modelos Teóricos: Leis de Amdahl e Gustafson . . 21
3.5.1 Lei de Gustafson: Escalabilidade Fraca e Diluição de Sobrecarga 22
3.6 Comparativo entre Paradigmas: Memória Compartilhada (OpenMP) vs. Memória Distribuída (MPI) 26

4 CONCLUSÃO 28

REFERÊNCIAS 31

ANEXO A—ENUNCIADO DO BENCHMARK DE MANDELBROT 32

1 INTRODUÇÃO

Este trabalho tem como objetivo desenvolver e avaliar o desempenho de diversas versões para o cálculo do Conjunto de Mandelbrot, seja sequencial ou paralela. Enquanto a Etapa 1 explorou paralelismo com uso de memória compartilhada com OpenMP, a Etapa 2 expande a investigação para o paradigma de memória distribuída utilizando a biblioteca MPI (Message Passing Interface).

O Conjunto de Mandelbrot é formado por números no plano complexo calculados a partir da fórmula $z_{n+1} = z_n^2 + c$, começando com $z_0 = 0$. Para saber se um ponto pertence ou não ao conjunto, foi usado o método de tempo de escape (escape-time): a fórmula é repetida até que o valor de $|z_n|$ ultrapasse o limite de corte (definido como 2 para esse projeto) ou até que se atinja um limite máximo de iterações (MAX_ITER, definido como 1000 para casos base nesse projeto). Indica-se o material (Mathigon, 2026) para um melhor entendimento da história e da matemática por trás do Conjunto de Mandelbrot.

Como os processos não compartilham a mesma memória, eles precisam trocar mensagens o tempo todo para organizar o trabalho e juntar a imagem final. Isso cria um problema claro: dividir o trabalho de forma estática quase não gasta tempo com mensagens, mas pode sobrecarregar alguns processos enquanto outros ficam sem fazer nada. Por outro lado, distribuir o trabalho de modo dinâmico divide as tarefas de forma equilibrada, mas aumenta o tempo gasto no envio de mensagens e pode sobrecarregar o processo principal que gerencia as tarefas.

Para avaliar como lidar com a comunicação e a divisão de tarefas em memória distribuída, foram criadas cinco abordagens no código:

i. **Divisão Estática em Blocos (`mpi_estatico` com `block`):** particiona a imagem em linhas de dimensão fixa distribuídos previamente entre os processos. Cada processo computa as linhas mapeadas ao seu *rank* e, ao término, a matriz global é reconstituída no processo 0 por meio de uma operação coletiva `MPI_Gatherv`.

ii. **Divisão Estática Cíclica (`mpi_estatico` com `cyclic`):** particiona as linhas da imagem de forma intercalada (*round-robin*), distribuindo as linhas de índice j conforme a regra de congruência modular do escalonador. Ao final do cálculo, os dados locais são transmitidos via `MPI_Gatherv` e reorganizados pelo processo 0 para remontar a ordem original das linhas.

iii. **Divisão Dinâmica Tradicional (`mpi_dinamico`):** adota o modelo Coordenador–Trabalhador (*Coordinator–Worker*) centralizado. O processo 0 atua exclusivamente

na gestão de uma fila de tarefas, despachando lotes de linhas inteiras (*chunks*) sob demanda através de mensagens pontual a pontual (`MPI_Send` e `MPI_Recv`), enquanto os processos de *ranks* 1 a $P-1$ realizam o cálculo dos pixels e retornam as iterações.

iv. **Divisão Dinâmica com Coordenador Participativo (`mpi_dinamico_participativo`):** estende o modelo dinâmico integrando o processo coordenador ao laço de computação de pixels. O processo 0 aloca fatias de trabalho para si próprio e, a cada linha local processada, realiza verificações não-bloqueantes com `MPI_Iprobe` para receber resultados e despachar novas tarefas aos processos trabalhadores que requisitarem.

v. **Divisão Dinâmica em Sub-Linha (`mpi_dinamico_meia_linha`):** altera a granularidade espacial das tarefas dinâmicas ao particionar cada linha horizontal da imagem em duas metades independentes (*half-rows*), totalizando $2 \times$ HEIGHT unidades de trabalho. O coordenador despacha essas frações de linhas via mensagens sob demanda para processamento pelos nós trabalhadores.

Para avaliar o impacto prático de cada estratégia no tempo total de processamento e na sobrecarga de comunicação, foram realizados experimentos sistemáticos variando o número de processos e as dimensões dos blocos de trabalho (*chunk size*). Essa exploração permite identificar qual arranjo responde de forma mais eficiente à heterogeneidade computacional intrínseca ao Conjunto de Mandelbrot em arquiteturas distribuídas.

Para quantificar e comparar o desempenho das abordagens, o estudo fundamenta-se em quatro métricas formais: o *Tempo de Execução*, que contabiliza a computação do fractal e a consolidação dos dados; o *Speedup*, referente à aceleração relativa obtida frente à versão sequencial de referência; a *Eficiência*, que expressa a taxa de aproveitamento efetivo dos núcleos computacionais alocados; e o *Fator de Balanceamento de Carga*, calculado a partir da dispersão temporal individual de cada processo para aferir a equidade da partição. Por fim, a escalabilidade observada é examinada sob as leis de Amdahl e Gustafson.

2 METODOLOGIA

2.1 Especificações e Parâmetros do Benchmark

As regras e os parâmetros adotados nesta etapa foram definidos com base no Anexo A, disponibilizado a toda a turma para garantir a padronização dos experimentos e a comparação justa entre os resultados.

O cálculo segue o método de tempo de escape (*escape-time*): para cada pixel obtém-se o número complexo c correspondente por mapeamento linear e repete-se a fórmula $z_{n+1} = z_n^2 + c$, partindo de $z_0 = 0$. O laço termina quando $|z|^2 > 4$ (equivalente a $|z| > 2$, evitando o custo de calcular raízes quadradas) ou quando se atinge o limite MAX_ITER. Todas as contas utilizam números de ponto flutuante em precisão dupla (`double`).

Os parâmetros oficiais e cenários de teste dividem-se em:

- **Cenário Padrão (Caso Base):**
  - Região no plano complexo: Re $\in [-2{,}0;\ 1{,}0]$ e Im $\in [-1{,}5;\ 1{,}5]$;
  - Resolução: 4096×4096 pixels;
  - Limite de iterações: $MAX_ITER = 1000$;
  - Critério de escape: $|z|^2 > 4$.
- **Casos de Escalabilidade e Estresse:**
  - *Escalabilidade Forte (Strong Scaling):* mantém a resolução fixa em 4096×4096 e varia a quantidade de processos;
  - *Escalabilidade Fraca (Weak Scaling):* aumenta proporcionalmente a resolução da imagem (por exemplo, 4096^2, 8192^2 e 16384^2) conforme mais recursos de processamento são adicionados;
  - *Desbalanceamento Acentuado:* região de aproximação (*zoom*) no "Vale dos Cavalos-Marinhos", com centro em $(-0{,}743643887;\ 0{,}131825904)$, largura de $3{,}0 \times 10^{-3}$ no eixo real (mantendo a proporção da tela) e $MAX_ITER = 5000$. Esse caso reúne uma grande quantidade de pixels de custo máximo para testar o comportamento das políticas de escalonamento.

A saída oficial é uma matriz com a contagem de iterações gravada em arquivo de texto plano (formato ASCII, com inteiros decimais em ordem *row-major* separados por espaço e terminados por quebras de linha), acompanhada de uma imagem PPM

para verificação visual. Para assegurar a fidelidade dos resultados obtidos pelas diferentes implementações paralelas sob a interface MPI, foi desenvolvida uma suíte automatizada de testes de corretude (`test_corretude_mpi.py`). O procedimento consistiu em comparar a matriz bidimensional de iterações de escape de cada versão paralela contra a matriz gerada pela versão sequencial canônica de referência (`ponto_a_ponto.cpp`). A validação foi realizada célula a célula por um verificador dedicado (`comparador.cpp`), inspecionando 262.144 pixels (resolução de 512×512) em dois cenários distintos: a Visão Completa do fractal e a região Vale dos Cavalos Marinhos. Foram testados todos os algoritmos MPI propostos, variando-se o tamanho de chunk (1, 4 e 16). Em todas as 78 configurações avaliadas, obteve-se correspondência estrita de 100%, confirmando que as técnicas de decomposição de domínio, redistribuição cíclica e coleta por mensagens preservaram a integridade da solução.

2.2 Implementação Sequencial de Referência

Como base para a aferição de *Speedup* e validação de corretude das implementações paralelas, utilizou-se a versão sequencial definitiva validada na Etapa 1 (`ponto_a_ponto.cpp`). O código foi desenvolvido em C++ e calcula o tempo de escape ponto a ponto sem atalhos geométricos, assegurando a fidelidade matemática tanto na visão global quanto em regiões assimétricas de aproximação. A saída do algoritmo foi homologada com zero divergências frente ao *script* canônico em Python fornecido para o benchmark.

O tempo computacional obtido por essa implementação (T_1) representa a linha de base para todos os cálculos de aceleração e eficiência em memória distribuída, garantindo que as métricas reflitam exclusivamente o custo computacional do laço do fractal.

2.3 Implementação Paralela com MPI

A versão paralela foi desenvolvida em C++ com a biblioteca MPI (*Message Passing Interface*) (Message Passing Interface Forum, 2021). Como os processos não compartilham memória entre si, cada um trabalha de forma isolada, sendo necessário dividir a imagem e trocar mensagens para juntar o resultado final no processo 0. No MPI, cada trabalhador aloca apenas a memória estritamente necessária para as linhas que está calculando, e apenas o processo 0 cria o espaço para guardar a matriz completa da imagem.

Para testar o impacto da comunicação e do equilíbrio de trabalho, foram desenvolvidas quatro implementações com estratégias diferentes:

2.3.1 Divisão Estática (`mpi_estatico.cpp`)

Nesta versão, a imagem é dividida antes de começar a conta e os processos calculam suas partes sem trocar mensagens durante a execução. Há duas opções no arquivo `in.txt`:

(a) **Blocos Contíguos (`block`):** corta a imagem em grandes blocos horizontais de mesmo tamanho. Cada processo calcula a sua faixa de linhas e salva em um vetor próprio. No fim, todos enviam suas partes para o processo 0 de uma só vez usando a função `MPI_Gatherv`.

(b) **Cíclica ou Intercalada (`cyclic`):** distribui as linhas pulando de forma alternada (em padrão *round-robin* ou por blocos definidos pelo parâmetro `CHUNK_SIZE`). Isso mistura linhas pesadas e linhas fáceis entre todos os núcleos. Ao final, os processos também enviam seus dados via `MPI_Gatherv`, e o processo 0 apenas organiza as linhas na ordem correta da imagem.

2.3.2 Divisão Dinâmica Tradicional (`mpi_dinamico.cpp`)

Para não depender de uma divisão fixa, foi criado o modelo Coordenador–Trabalhador (*Coordinator–Worker*). Aqui, o processo 0 atua apenas como organizador:

- No começo, o coordenador manda uma mensagem (`TAG_WORK`) para cada trabalhador (processos 1 a $P-1$) com um bloco de linhas para calcular (`CHUNK_SIZE`);
- Quando um trabalhador termina, ele envia os pontos calculados de volta (`TAG_RESULT`);
- O processo 0 recebe esses pontos via `MPI_Recv`, copia diretamente na imagem final e manda na hora mais uma tarefa livre da fila;
- Quando não há mais linhas sobrando, o coordenador envia uma mensagem de parada (`TAG_STOP`) para dispensar os trabalhadores.

2.3.3 Divisão Dinâmica com Coordenador Participativo (`mpi_dinamico_participativo.cpp`)

O modelo tradicional equilibra bem o esforço dos trabalhadores, mas deixa o processo 0 ocioso sem calcular pixels. Em um computador comum de 4 núcleos, isso desperdiça 25% da máquina. Para resolver isso, implementou-se o Coordenador Participativo.

Nesta versão, o processo 0 distribui as primeiras tarefas, mas também pega fatias de linhas para ele mesmo calcular. Para não deixar os outros núcleos parados esperando trabalho, o processo 0 usa a função `MPI_Iprobe` a cada linha que calcula. Se perceber que algum trabalhador terminou e devolveu resultados, ele pausa seu cálculo por um instante,

recebe os dados, despacha uma nova tarefa para esse nó e volta a calcular seus próprios pontos.

2.3.4 Divisão Dinâmica em Meia-Linha (`mpi_dinamico_meia_linha.cpp`)

Buscando uma divisão ainda mais fina (semelhante ao efeito do `collapse` feito com OpenMP), esta versão divide cada linha horizontal ao meio, gerando o dobro de tarefas ($2 \times HEIGHT$).

Dessa forma, o coordenador pode entregar a metade esquerda de uma linha para um processo e a metade direita para outro, evitando que uma única linha difícil sobrecarregue um trabalhador inteiro enquanto os demais ficam ociosos.

2.4 Casos de Testes e Configurações Avaliadas

Para avaliar a influência da partição de dados e do envio de mensagens sobre a execução do benchmark, foram definidas as configurações de teste para cada estilo de paralelização sintetizadas na Tabela 1.

As 13 configurações apresentadas na Tabela 1 foram avaliadas nas resoluções de grade 4096×4096, 8192×8192 e 16384×16384 pixels, abrangendo os cenários de Visão Global e do Vale dos Cavalos-Marinhos. Para todos os ensaios paralelos foram realizadas 5 repetições independentes para cada arranjo. O fluxo foi automatizado pelo *script* em Python `run_tests.py`, que disparou as execuções e consolidou as saídas para prevenir ensaios redundantes.

Com o objetivo de atenuar ruídos experimentais e oscilações do sistema operacional, avaliou-se a dispersão das 5 execuções de cada cenário por meio do cálculo do desvio-padrão amostral. Para a definição do tempo representativo oficial de cada ensaio, comparou-se o comportamento da mediana com o da média aritmética dos três valores centrais (descartando os tempos extremos de mínimo e máximo), analisando qual dessas métricas oferece maior estabilidade e fidelidade estatística frente a eventuais *outliers* de latência de comunicação.

2.5 Ambiente de Execução e Hardware

Os experimentos de desempenho foram executados em um computador portátil com processador Intel Core i3-8130U, composto por 2 núcleos físicos e 4 núcleos lógicos via tecnologia *Hyper-Threading*, com frequência base de 2,20 GHz e frequência máxima sob *Turbo Boost* de até 3,40 GHz. A memória *cache* é estruturada em 128 KB de L1,

Tabela 1 – Configurações experimentais de escalonamento e granularidade em MPI.

Estratégia / Estilo	***Chunk***	**Descrição do Particionamento e Comunicação**
Estático em Bloco	0	Cada processo recebe uma fatia contínua de linhas de tamanho $\lfloor HEIGHT/P \rfloor$, realizando uma única coleta via `MPI_Gatherv` ao término do cálculo.
Estático Cíclico	1	Intercalação unitária de linhas em padrão *round-robin* ($j \pmod P == \text{rank}$).
	2	Distribuição intercalada em blocos de 2 linhas contíguas.
	4	Distribuição intercalada em blocos de 4 linhas contíguas.
Dinâmico Dedicado	1	Envio sob demanda de 1 linha inteira por mensagem de tarefa.
	2	Envio sob demanda de lotes com 2 linhas inteiras agrupadas.
	4	Envio sob demanda de lotes com 4 linhas inteiras agrupadas.
Dinâmico Participativo	1	Processamento local e despacho sob demanda configurados para 1 linha por tarefa.
	2	Processamento local e despacho em lotes de 2 linhas.
	4	Processamento local e despacho em lotes de 4 linhas.
Dinâmico Meia-Linha	1	Partição de cada linha horizontal ao meio, despachando 1 meia-linha por tarefa ($2 \times HEIGHT$ tarefas totais).
	2	Despacho sob demanda de blocos com 2 meias-linhas agrupadas.
	4	Despacho sob demanda de blocos com 4 meias-linhas agrupadas.

512 KB de L2 e 4 MB de cache L3 compartilhado. O sistema dispõe de 12 GB de memória principal RAM DDR4 operando em *dual-channel*.

O ambiente de software adotado foi a distribuição Linux Zorin OS 18.1 Core (base Ubuntu/Debian), com *kernel* Linux de 64 bits. Os programas foram compilados em C++ utilizando o compilador `mpic++` com suporte à biblioteca OpenMPI (versão 4.1.6). Para maximizar o desempenho computacional, foram aplicadas as diretivas de otimização agressiva `-O3` e `-ffast-math`.

Para a caracterização teórica do poder computacional, calculou-se o desempenho máximo teórico em ponto flutuante (R_{peak}). Considerando a extensão vetorial AVX2 acoplada à unidade FMA3, a arquitetura é capaz de executar 16 operações em ponto flutuante de dupla precisão (FP64) por ciclo em cada núcleo físico:

$$R_{\text{peak}} = N_{\text{núcleos}} \times f \times \text{FLOPs/ciclo} \tag{2.1}$$

Em regime de frequência base (2,20 GHz), a capacidade computacional atinge $R_{\text{peak}} = 2 \times 2{,}20 \times 16 = 70{,}4$ GFLOPS. Já sob frequência máxima com *Turbo Boost* (3,40 GHz), o teto alcança 108,8 GFLOPS. Os testes foram executados com o perfil de energia para desempenho, mas sem o *Turbo Boost* ativado.

2.6 Procedimentos de Medição de Tempo e Métricas de Avaliação

A tomada de tempo abrange estritamente o cálculo das iterações e a comunicação necessária para a consolidação dos dados entre os processos. Tarefas sequenciais de I/O em disco, como alocação inicial de estruturas, renderização gráfica e gravação de arquivos, foram inteiramente isoladas para evitar distorções nas medições de paralelismo.

A contagem temporal utilizou a função de alta precisão `MPI_Wtime()`, sincronizando o início por meio de `MPI_Barrier`. Nas versões com divisão estática, utilizou-se a redução coletiva `MPI_Reduce` com operador `MPI_MAX` para computar o tempo final decorrido até a conclusão do processo mais tardio. Adicionalmente, todos os códigos realizam a coleta dos tempos úteis de cálculo individual de cada nó através de `MPI_Gather`, gravando os dados no arquivo `tempos_processos_mpi.csv`.

Para a avaliação quantitativa do desempenho, empregaram-se as seguintes métricas formais:

- **Tempo de Execução (T_p):** Duração (em segundos) da computação paralela empregando p processos, adotando-se o tempo T_1 obtido na versão sequencial como referência basal.

- ***Speedup*** **(S_p):** Expressa o ganho relativo de velocidade em relação à referência sequencial:

$$S_p = \frac{T_1}{T_p} \tag{2.2}$$

- **Eficiência (E_p):** Mede o percentual de aproveitamento efetivo dos processadores alocados:

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p} \tag{2.3}$$

- **Fator de Balanço de Carga (F_{LB}):** Quantifica a razão de sincronização e a dispersão temporal entre os processos, definido pela diferença relativa entre o tempo do último processo a terminar ($T_{\max}$) e o primeiro ($T_{\min}$):

$$F_{LB} = \frac{T_{\max} - T_{\min}}{T_{\max}} \tag{2.4}$$

  Valores próximos a 0,0 indicam condição ótima ($T_{\max} \approx T_{\min}$), enquanto valores próximos a 1,0 indicam forte desbalanceamento com espera excessiva ($T_{\max} \gg T_{\min}$).

3 RESULTADOS E DISCUSSÃO

Antes da análise comparativa de desempenho, todas as saídas geradas pelas implementações paralelas foram submetidas ao protocolo de validação descrito na Seção 2.1, obtendo-se 100% de conformidade numérica frente à referência sequencial. Uma vez assegurada a corretude algorítmica, as seções seguintes concentram-se na avaliação temporal, escalabilidade e balanceamento de carga entre as abordagens avaliadas.

3.1 Definição do Tempo Representativo de Referência

Antes de proceder à análise comparativa de desempenho, avaliou-se a dispersão temporal das execuções para determinar qual métrica estatística de tendência central representaria com maior fidelidade o comportamento determinístico dos algoritmos, mitigando interferências e ruídos do sistema operacional.

Para quantificar essa variabilidade de forma padronizada, adotou-se o cálculo do desvio-padrão amostral (s) e do Coeficiente de Variação ($CV\%$). O desvio-padrão baseia-se em $k-1$ graus de liberdade (correção de Bessel), sendo k o número de repetições consideradas para a métrica avaliada ($k = 5$ para a média global e $k = 3$ para as médias truncadas ou originárias de amostras menores):

$$s = \sqrt{\frac{1}{k-1}\sum_{i=1}^{k}\left(t_i - \bar{t}\right)^2} \tag{3.1}$$

Para avaliar a variabilidade relativa percentual de cada estimador de maneira proporcional à sua própria magnitude, utilizou-se o Coeficiente de Variação:

$$CV = \left(\frac{s}{\hat{t}}\right) \times 100\% \tag{3.2}$$

em que $\hat{t}$ representa o estimador pontual adotado (seja a média global, a média truncada ou a mediana).

Munido dessas métricas para a tomada de decisão, confrontou-se o comportamento da mediana amostral com o da média aritmética sob dois regimes amostrais distintos: a base ampla com 5 repetições (aplicada aos ensaios MPI e sequenciais) e o conjunto comparativo da etapa anterior em OpenMP com 3 repetições. A Tabela 2 sintetiza os parâmetros de dispersão observados ao longo da base experimental de 390 execuções.

A análise técnica e estatística fundamenta a escolha das métricas oficiais de acordo com o tamanho da amostra avaliada. Nas baterias de teste com 5 repetições ($N = 5$), a escolha pela média dos três tempos do meio apoia-se em três fatores centrais. Primeiro,

Tabela 2 – Comparativo estatístico entre os estimadores de tempo de execução

Estimador e Amostra	**Desvio-Padrão Médio**	**CV Médio (%)**
Média Global ($N = 5$)	Base (100%)	$\approx 6{,}58\%$
Mediana ($N = 5, t_{(3)}$)	$\approx -30\%$	$\approx 4{,}12\%$
Média Truncada ($N = 5$, 3 centrais)	$\approx -50\%$	$\approx 3{,}26\%$
Média dos 3 Tempos ($N = 3$)	**Base (100%)**	$\approx 5{,}37\%$
Mediana ($N = 3, t_{(2)}$)	$+11{,}6\%$	$\approx 6{,}30\%$

ela remove distorções causadas pelo ambiente de execução, enquanto o ensaio mais lento frequentemente sofre com interrupções do *kernel*, faltas de página ou concorrência em segundo plano. O descarte dos dois extremos atenua esses ruídos e diminui o desvio-padrão amostral em cerca de 50%. Segundo, essa estimativa é mais eficiente que a mediana simples, pois, ao agregar a informação de três medições homogêneas em vez de fixar-se em um único ponto discreto ($t_{(3)}$), reduz o erro padrão em aproximadamente 20% conforme o Teorema Central do Limite. Terceiro, o Coeficiente de Variação (CV) médio contrai-se de 6,58% (considerando as cinco rodadas) para 3,26% (com as três do meio), garantindo alta consistência interna para os cálculos.

Por outro lado, nas análises do OpenMP com apenas 3 repetições ($N = 3$), o descarte de dois extremos eliminaria dois terços dos dados experimentais. Isso reduziria a amostra a um único valor, inviabilizando o cálculo da dispersão. Nesse cenário, a avaliação confirmou que a média dos três tempos supera a mediana ($t_{(2)}$) em 100% dos casos avaliados. Devido à propriedade dos mínimos quadrados, a média aritmética minimiza a soma dos desvios, resultando em um desvio-padrão base inferior. Essa estabilidade é comprovada pela contração do Coeficiente de Variação (CV) médio, que cai de 6,30% na mediana para 5,37% com a média integral. Embora a diferença relativa média entre os tempos nominais de ambas as métricas seja de apenas 2,26%, o que atesta uma forte proximidade, a média apresenta clara vantagem na eficiência e precisão estatística.

Dessa forma, padronizou-se a média truncada dos três tempos centrais para as baterias de 5 repetições e a média aritmética integral para as amostras de 3 repetições como o tempo representativo oficial (T_p) adotado em todas as discussões, tabelas e gráficos subsequentes.

Como linha de base sequencial canônica (T_1), utilizou-se a versão `ponto_a_ponto_v2`. Essa implementação preserva estritamente o método de tempo de escape ponto a ponto na imagem, mantendo a integridade matemática sem atalhos geométricos, mas com pequenas melhorias e simplificações em relação a versão `ponto_a_ponto` utilizada na Etapa 1. A Tabela 3 apresenta os tempos de referência sequenciais tratados (T_1), acompanhados de seus respectivos desvios e coeficientes de variação obtidos após o expurgo dos extremos amostrais.

Tabela 3 – Tempos de referência sequenciais oficiais da versão `ponto_a_ponto_v2` (T_1)

Cenário	**Resolução**	T_1 **Tratado Oficial**	**Desvio** (s)	**CV (%)**
Visão Completa	4096 × 4096	**11,0968 s**	±0,0289 s	0,26%
Visão Completa	8192 × 8192	**43,9691 s**	±0,0327 s	0,07%
Visão Completa	16384 × 16384	**175,7080 s**	±0,0281 s	0,016%
Zoom Seahorse	4096 × 4096	**4,7773 s**	±0,0044 s	0,09%
Zoom Seahorse	8192 × 8192	**19,0409 s**	±0,0038 s	0,02%
Zoom Seahorse	16384 × 16384	**76,1291 s**	±0,0507 s	0,06%

3.2 Discussão dos Resultados: Caso 1 (Visão Global)

No Caso 1, analisamos a Visão Global do fractal de Mandelbrot na região com Re $\in [-2{,}0;\ 1{,}0]$ e Im $\in [-1{,}5;\ 1{,}5]$, fixando o limite em 1000 iterações. A distribuição estática cíclica (`cyclic`) foi a mais rápida em todas as resoluções avaliadas, estabelecendo os menores tempos e os maiores índices de aceleração da bateria de testes. Na resolução máxima de 16K, ela completou o processamento em apenas 80,27 s com *chunk* 4, alcançando um *Speedup* de 2,19× e uma eficiência sobre os dois núcleos físicos de 109,4%. Esse sucesso decorre da partição matemática das linhas realizada antes do cálculo, que dispensa a troca de mensagens intermediárias e deixa a sincronização restrita à coleta final via `MPI_Gatherv`. Além disso, ao intercalar as linhas ao longo da imagem, cada processo recebeu uma fatia equilibrada tanto das bordas vazias de escape rápido quanto do centro denso da cardioide principal. A eficiência física acima de 100% comprova que o *Hyper-Threading* foi aproveitado em sua capacidade plena, mantendo as unidades funcionais dos 2 núcleos físicos ocupadas durante todo o laço. A Tabela 4 detalha as métricas obtidas por essa abordagem para as três resoluções.

Tabela 4 – Desempenho da divisão estática cíclica (`cyclic`) frente ao sequencial ($p = 4$).

Resolução	***Chunk***	**Tempo** T_p **(s)**	***Speedup*** (S_p)	$E_{p=4}$	$E_{\textbf{físico}}$
4096 × 4096 (4K)	1	**5,55**	**2,00×**	**50,0%**	**100,0%**
	2	**5,55**	**2,00×**	**50,0%**	**100,0%**
	4	5,62	1,97×	49,4%	98,7%
8192 × 8192 (8K)	1	22,27	1,97×	49,4%	98,7%
	2	22,08	1,99×	49,8%	99,6%
	4	**21,60**	**2,04×**	**50,9%**	**101,8%**
16384 × 16384 (16K)	1	81,84	2,15×	53,7%	107,3%
	2	80,66	2,18×	54,5%	108,9%
	4	**80,27**	**2,19×**	**54,7%**	**109,4%**

Linhas de base: $T_1 = 11{,}0968$ s (4K), 43,9691 s (8K) e 175,7080 s (16K). $E_{\text{físico}} = S_p/2$.

Em contrapartida, a divisão estática por blocos contíguos (`block`) apresentou um colapso de desempenho, resultando em perda de rendimento em todas as resoluções avaliadas, conforme sintetizado na Tabela 5. Esse gargalo foi provocado pela geometria

da Visão Global: os cálculos pesados concentram-se estritamente nas faixas centrais da grade. Ao fatiar a matriz em quatro grandes blocos fixos contínuos, os processos das pontas (rank 0 e 3) terminaram suas áreas quase que instantaneamente e ficaram bloqueados aguardando os processos centrais (rank 1 e 2). O fato de a eficiência física ter ficado abaixo de 50% comprova que, na prática, mais da metade da capacidade da CPU permaneceu ociosa aguardando os nós centrais.

Tabela 5 – Desempenho da divisão estática por blocos contíguos (`block`) ($p = 4$).

Resolução	***Chunk***	**Tempo T_p (s)**	***Speedup* (S_p)**	$E_{p=4}$	$E_{\textbf{físico}}$
4096×4096 (4K)	N/p	11,37	0,98×	24,4%	48,8%
8192×8192 (8K)	N/p	**47,96**	**0,92×**	**22,9%**	**45,8%**
16384×16384 (16K)	N/p	180,51	0,97×	24,3%	48,7%

As estratégias dinâmicas com coordenador dedicado (`dynamic`) e com divisão em meia-linha (`half-row`) conseguiram contornar o desbalanceamento espacial do fractal, mas encontraram um teto estrutural de aceleração. Conforme detalhado na Tabela 6, ambas estabilizaram seus ganhos com *Speedup* entre 1,35× e 1,45×, alcançando eficiência física entre 67,4% e 72,6%. Esse limite ocorre porque o rank 0 atua unicamente como despachante e não calcula nenhum pixel. Logo, com 4 processos alocados, apenas 3 realizam computação útil, o que impõe um teto teórico de aproveitamento de $3/4 = 75\%$. Vale notar que, em cenários com uma quantidade significativamente maior de processos além do *rank* 0 (como em nós com mais núcleos ou em clusters de grande porte), é razoável projetar um rendimento global superior para esses esquemas dinâmicos. Nesses arranjos, a proporção de processos computando pixels frente ao nó de controle aumentaria consideravelmente, diluindo o peso do coordenador dedicado e fazendo com que o processo 0 passe muito menos tempo ocioso aguardando requisições, mesmo continuando sem calcular pixels diretamente. Em relação ao tamanho das mensagens, despachar 1 linha por vez revelou-se a escolha mais lenta devido ao excesso de pacotes na rede (em 16K, 124,99 s contra cerca de 121 s nos *chunks* 2 e 4). Agrupar o envio em lotes de 2 ou 4 linhas amorteceu esse custo de controle e reduziu a espera no coordenador.

Por fim, o modelo com coordenador participativo (`dynamic_part`) apresentou uma falha crítica de desempenho, tornando-se cerca de duas vezes e meia mais lento que a própria execução sequencial (436,27 s contra 175,71 s em 16K, resultando em *Speedup* de apenas 0,40× e eficiência física de 20,1%). O objetivo dessa versão era colocar o processo 0 para calcular tarefas enquanto aguardava os demais, mas o cálculo de Mandelbrot exige esforço contínuo de processamento. Assim, enquanto o coordenador executava seus próprios pixels, ele não atendia com a devida rapidez às mensagens de retorno via `MPI_Iprobe`. Essa retenção deixou os três processos trabalhadores longos períodos parados sem tarefas para processar, quebrando a concorrência e demonstrando que a sobrecarga de gerência concorrente superou amplamente as vantagens do dinamismo.

Tabela 6 – Comparativo das abordagens dinâmicas sob demanda para 4 processos MPI.

Estratégia	**Resolução**	***Chunk***	**Tempo T_p (s)**	***Speedup*** **(S_p)**	$E_{p=4}$	$E_{\textbf{físico}}$
`dynamic`	8192 × 8192 (8K)	1	32,61	1,35×	33,7%	67,4%
		2	31,82	1,38×	34,5%	69,1%
		4	31,65	1,39×	34,7%	69,5%
	16384 × 16384 (16K)	1	124,99	1,41×	35,1%	70,3%
		2	**121,05**	**1,45×**	**36,3%**	**72,6%**
		4	121,14	1,45×	36,3%	72,5%
`half-row`	8192 × 8192 (8K)	1	31,46	1,40×	34,9%	69,9%
		2	**30,42**	**1,45×**	**36,1%**	**72,3%**
		4	30,67	1,43×	35,8%	71,7%
	16384 × 16384 (16K)	1	124,03	1,42×	35,4%	70,8%
		2	123,23	1,43×	35,6%	71,3%
		4	122,76	1,43×	35,8%	71,6%
`dynamic_part`	8192 × 8192 (8K)	1	107,00	0,41×	10,3%	20,5%
		2	107,67	0,41×	10,2%	20,4%
		4	106,99	0,41×	10,3%	20,5%
	16384 × 16384 (16K)	1	431,89	0,41×	10,2%	20,3%
		2	**437,14**	**0,40×**	**10,0%**	**20,1%**
		4	436,27	0,40×	10,1%	20,1%

3.3 Discussão dos Resultados: Caso 2 (Vale dos Cavalos-Marinhos)

No Caso 2, avaliamos a região de aproximação (*zoom*) no Vale dos Cavalos-Marinhos (*Seahorse Valley*), com centro em (−0,743643887; 0,131825904), largura de $3{,}0 \times 10^{-3}$ no eixo real e limite estendido para 5000 iterações. A distribuição estática cíclica (`cyclic`) manteve os menores tempos absolutos e os maiores índices de aceleração entre todas as resoluções avaliadas. Na resolução de 16K, ela completou o processamento em apenas 39,50 s com *chunk* 1, alcançando um *Speedup* de 1,93× e uma eficiência sobre os dois núcleos físicos de 96,4%. Esse resultado evidencia a eficácia da partição intercalada em balancear a densidade de iterações do fractal sem introduzir sobrecargas de comunicação durante a computação, restringindo a sincronização à coleta final via `MPI_Gatherv`. A Tabela 7 detalha as métricas obtidas por essa abordagem para as três resoluções.

Diferentemente do Caso 1, a divisão estática por blocos contíguos (`block`) não colapsou no Vale dos Cavalos-Marinhos, registrando acelerações crescentes com o aumento da grade (1,42× em 4K, 1,61× em 8K e 1,68× em 16K, alcançando 83,9% de eficiência física), conforme sintetizado na Tabela 8. Embora ainda ocorra desbalanceamento entre as faixas, a dispersão das espirais ao longo da área de ampliação evitou que os processos das extremidades ficassem completamente ociosos, garantindo ganho efetivo frente à execução sequencial.

As estratégias dinâmicas com coordenador dedicado (`dynamic`) e com partição em

Tabela 7 – Desempenho da divisão estática cíclica (`cyclic`) no Seahorse frente ao sequencial ($p = 4$).

Resolução	***Chunk***	**Tempo T_p (s)**	***Speedup* (S_p)**	$E_{p=4}$	$E_{\textbf{físico}}$
4096 × 4096 (4K)	1	3,23	1,48×	36,9%	73,8%
	2	**3,14**	**1,52×**	**38,1%**	**76,1%**
	4	3,22	1,48×	37,1%	74,2%
8192 × 8192 (8K)	1	10,75	1,77×	44,3%	88,6%
	2	**10,37**	**1,84×**	**45,9%**	**91,8%**
	4	10,90	1,75×	43,7%	87,3%
16384 × 16384 (16K)	1	**39,50**	**1,93×**	**48,2%**	**96,4%**
	2	40,48	1,88×	47,0%	94,0%
	4	39,85	1,91×	47,8%	95,5%

Linhas de base: $T_1 = 4{,}78\,\text{s}$ (4K), $19{,}04\,\text{s}$ (8K) e $76{,}13\,\text{s}$ (16K). $E_{\text{físico}} = S_p/2$.

Tabela 8 – Desempenho da divisão estática por blocos contíguos (`block`) no Seahorse ($p = 4$).

Resolução	***Chunk***	**Tempo T_p (s)**	***Speedup* (S_p)**	$E_{p=4}$	$E_{\textbf{físico}}$
4096 × 4096 (4K)	N/p	3,35	1,42×	35,6%	71,2%
8192 × 8192 (8K)	N/p	11,83	1,61×	40,2%	80,5%
16384 × 16384 (16K)	N/p	**45,37**	**1,68×**	**41,9%**	**83,9%**

meia-linha (`half-row`) apresentaram comportamentos distintos diante da alta densidade de cálculo. A Tabela 9 reúne os dados comparativos dessas abordagens sob demanda. O esquema `dynamic` tradicional estabilizou seu ganho em 1,37× em 16K (eficiência física de 68,5%) com *chunk* 2, limitado estruturalmente pela dedicação exclusiva do processo 0 ao despacho. Por outro lado, a decomposição em meia-linha (`half-row`) obteve ganho expressivo nas resoluções mais altas: em 16K, alcançou $S_p = 1{,}72\times$ e eficiência física de 85,9% no *chunk* 2 (44,33 s), superando inclusive a divisão em blocos contíguos. A granularidade mais fina da meia-linha permitiu que duas metades de uma mesma linha densa fossem divididas entre trabalhadores distintos, absorvendo com eficácia as assimetrias locais do fractal. Em ambas as estratégias, o envio com *chunk* 1 apresentou ligeira desvantagem em relação a *chunks* 2 e 4 devido ao maior tráfego de mensagens na interface de rede.

Por fim, o modelo com coordenador participativo (`dynamic_part`) repetiu o colapso de desempenho observado no caso anterior, operando em regime de desaceleração extrema em todas as resoluções (S_p entre 0,33× e 0,40× e eficiência física inferior a 21%). Em 16K, o tempo saltou para mais de 211 s contra os 76,13 s da referência sequencial. Como o teto de iterações do Seahorse foi quintuplicado para 5000 iterações, o custo aritmético de cada linha processada localmente pelo processo 0 aumentou substancialmente. Consequentemente, o intervalo entre as sondagens via `MPI_Iprobe` ampliou-se, agravando a

Tabela 9 – Comparativo das abordagens dinâmicas sob demanda no Seahorse ($p = 4$).

Estratégia	**Resolução**	***Chunk***	**Tempo T_p (s)**	***Speedup* (S_p)**	$E_{p=4}$	$E_{\textbf{físico}}$
`dynamic`	8192 × 8192 (8K)	1	14,62	1,30×	32,6%	65,1%
		2	14,10	1,35×	33,8%	67,5%
		4	14,23	1,34×	33,5%	66,9%
	16384 × 16384 (16K)	1	56,39	1,35×	33,7%	67,5%
		2	55,53	1,37×	34,3%	68,5%
		4	57,91	1,31×	32,9%	65,7%
`half-row`	8192 × 8192 (8K)	1	15,05	1,26×	31,6%	63,2%
		2	14,24	1,34×	33,4%	66,9%
		4	14,45	1,32×	32,9%	65,9%
	16384 × 16384 (16K)	1	44,76	1,70×	42,5%	85,0%
		2	**44,33**	**1,72×**	**42,9%**	**85,9%**
		4	44,61	1,71×	42,7%	85,3%
`dynamic_part`	8192 × 8192 (8K)	1	48,45	0,39×	9,8%	19,6%
		2	48,39	0,39×	9,8%	19,7%
		4	47,23	0,40×	10,1%	20,2%
	16384 × 16384 (16K)	1	**227,74**	**0,33×**	**8,4%**	**16,7%**
		2	211,71	0,36×	9,0%	18,0%
		4	220,60	0,35×	8,6%	17,3%

retenção de mensagens dos trabalhadores e deixando os nós remotos ociosos por períodos prolongados (*worker starvation*), o que confirma a inadequação dessa arquitetura para laços de computação contínua e intensiva.

3.4 Análise do Desbalanceamento de Carga

Para quantificar a dispersão temporal entre os processos e verificar a ocorrência de ociosidade na sincronização final, adotou-se o Fator de Balanço de Carga (F_{LB}), formalizado na Equação (2.4). Valores próximos a 0,0 (0%) indicam balanceamento ideal ($T_{\max} \approx T_{\min}$), enquanto valores tendendo a 1,0 (100%) traduzem assimetria severa e espera ociosa na barreira coletiva.

No cálculo empírico do F_{LB}, é indispensável distinguir o papel arquitetural de cada rank no MPI. Nas estratégias estáticas e no dinâmico participativo, todos os $p = 4$ processos computam ativamente frações do conjunto fractal, fazendo com que o domínio de avaliação abranja a totalidade dos ranks (R_0, R_1, R_2, R_3). Por outro lado, nas abordagens dinâmicas com coordenador dedicado (`dynamic` e `half-row`), o rank R_0 atua exclusivamente na infraestrutura de controle, despacho e recepção de mensagens ($T_{R0} \approx 0{,}00\,\text{s}$). A inclusão direta desse nó na Equação (2.4) resultaria em um desbalanço artificial de $F_{LB} = 100\%$ devido ao tempo nulo de cálculo, mascarando a real eficiência do escalonamento. Dessa

forma, para os esquemas de mestre dedicado, a métrica é aplicada estritamente sobre o conjunto de processos trabalhadores computacionais (R_1, R_2, R_3), quantificando com exatidão a capacidade do coordenador em balancear a carga entre os nós de processamento.

A Tabela 10 consolida os tempos individuais tratados via média truncada ($N = 5$) e os fatores F_{LB} resultantes na resolução de 16384 × 16384 (16K).

Tabela 10 – Diagnóstico de tempos por rank e Fator de Balanço de Carga F_{LB} (16384^2, $p = 4$).

Cenário	**Estratégia**	***Chunk***	T_{R0} **(s)**	T_{R1} **(s)**	T_{R2} **(s)**	T_{R3} **(s)**	$T_{\max}$ **(s)**	$T_{\min}$ **(s)**	F_{LB} **(%)**
Visão Completa	`block`	N/p	4,59	178,65	179,16	4,53	179,16	4,53	97,5%
	`cyclic`†	4	77,99	77,85	75,34	76,37	**77,99**†	75,34	3,4%
	`dynamic`	2	0,00	118,76	118,72	118,77	118,77	118,72	**0,0%**
	`dynamic_part`	2	435,23	0,00	0,00	0,00	435,23	0,00	**100,0%**
	`half-row`	2	0,00	120,41	120,47	120,40	120,47	120,40	0,1%
Seahorse (Zoom)	`block`	N/p	34,36	23,87	45,17	40,00	45,17	23,87	47,2%
	`cyclic`†	4	35,71	36,91	37,11	36,29	**37,11**†	35,71	3,8%
	`dynamic`	2	0,00	53,06	53,05	53,07	53,07	53,05	**0,0%**
	`dynamic_part`	2	208,84	0,10	0,04	0,06	208,84	0,04	**100,0%**
	`half-row`	2	0,00	42,85	42,85	42,86	42,86	42,85	**0,0%**

† Estratégia vencedora em tempo total de execução ($T_{\max}$) para o respectivo cenário.

A comparação dos fatores F_{LB} comprova quantitativamente os gargalos e as vantagens relativas de cada arquitetura, evidenciando uma lição fundamental de computação de alto desempenho: a estratégia mais rápida não foi a que zerou o desbalanceamento, e atingir $F_{LB} \approx 0{,}0\%$ não traduz necessariamente a melhor solução algorítmica.

A distribuição estática cíclica com granularidade de 4 linhas consolidou-se como a mais rápida e eficiente da aplicação (menores tempos globais $T_{\max} = 77{,}99$ s na Visão Completa e 37,11 s no Seahorse), mantendo um desbalanceamento residual muito baixo ($F_{LB} = 3{,}4\%$ e $3{,}8\%$). Ao intercalar faixas regulares, ela dissolveu as fronteiras densas do fractal igualmente entre todos os processos sem incorrer em nenhuma mensagem extra de controle durante a execução. O pequeno resíduo de dispersão temporal (2,65 s) foi amplamente compensado pelo custo nulo de sincronização intermediária e pelo uso integral de todos os núcleos de processamento ($p = 4$), viabilizando os maiores índices de aceleração e eficiência física.

Em contrapartida, as abordagens sob demanda `dynamic` e `half-row` foram as únicas que de fato zeraram o desbalanceamento entre os executores ($F_{LB} = 0{,}0\%$ e $0{,}1\%$), mantendo desvios de meros milissegundos entre R_1, R_2 e R_3. Contudo, esse equilíbrio milimétrico não as tornou as mais rápidas do sistema. O tempo de execução dessas soluções foi cerca de 50% superior ao da abordagem cíclica (demandando mais de 118 s a 120 s na Visão Completa e até 53 s no Seahorse). Esse contraste demonstra que zerar o balanceamento de carga é apenas um dos componentes de desempenho: o custo estrutural de reservar compulsoriamente 25% do hardware (R_0) exclusivamente para tarefas de

coordenação e a latência de troca de mensagens limitaram severamente o rendimento global, comprovando que a equalização estrita de carga não supera o impacto do desperdício de núcleos computacionais em sistemas com poucos processos.

No extremo oposto, a divisão estática por blocos contíguos expôs o colapso por desbalanceamento estrutural. Na Visão Completa, os ranks R_0 e R_3 receberam apenas regiões periféricas de escape rápido, concluindo suas fatias em cerca de 4,5 s, enquanto R_1 e R_2 concentraram a área densa da cardioide ($Y \in [-0{,}6; 0{,}6]$). A diferença absoluta atingiu $\Delta T = 174{,}63\,\text{s}$ ($F_{LB} = 97{,}5\%$), mantendo os processos periféricos ociosos na barreira do `MPI_Gatherv` por mais de 97% do tempo e limitando a eficiência física à metade ($E_{\text{físico}} \approx 48{,}7\%$). No caso Seahorse, a dispersão foi moderada pela maior densidade média do zoom, mas manteve assimetria expressiva ($F_{LB} = 47{,}2\%$).

Por fim, a estratégia com mestre participativo (`dynamic_part`) combinou o pior dos dois cenários, registrando desbalanceamento total ($F_{LB} = 100{,}0\%$). A tentativa de alocar esforço de cálculo ao coordenador provocou atrasos críticos no atendimento de mensagens via `MPI_Iprobe`. Os processos trabalhadores consumiram seus primeiros lotes em frações de segundo ($T_{R1,R2,R3} \leq 0{,}10\,\text{s}$) e entraram em inanição crônica (*worker starvation*), enquanto o Rank 0 processou isoladamente quase a totalidade da grade (435,23 s e 208,84 s), anulando o paralelismo da aplicação.

3.5 Escalabilidade e Modelos Teóricos: Leis de Amdahl e Gustafson

A interpretação da capacidade de aceleração e dos limites físicos do paralelismo no cálculo do fractal de Mandelbrot é regida por dois modelos clássicos de computação de alto desempenho: a Lei de Amdahl e a Lei de Gustafson. A distinção teórica entre os modelos reside na premissa adotada sobre o comportamento do volume de trabalho diante do acréscimo de recursos computacionais.

A avaliação da escalabilidade forte sob a ótica da Lei de Amdahl revela como a fração não paralelizável e os limites arquiteturais governam o ganho prático de desempenho ao variar as unidades de processamento de $p = 1$ até $p = 4$ com o tamanho de grade fixo.

Nas abordagens dinâmicas com coordenador dedicado (`dynamic` e `half-row`), a raiz do gargalo é conceitual: o processo 0 limita-se à orquestração de mensagens, despacho de tarefas e recepção de parcelas, não realizando cálculo útil de pixels. Em $p = 2$, esse modelo apresenta ganho quase nulo frente ao sequencial, mantendo $S_2 \approx 0{,}98\times$ na Visão Completa e entre $0{,}93\times$ e $0{,}97\times$ no Seahorse em todas as resoluções, dado que apenas 1 trabalhador executa computação de fato. Ao expandir para $p = 3$ e $p = 4$, a adição de nós trabalhadores eleva o desempenho, porém as curvas encontram um platô assintótico rígido. Em $p = 4$, onde apenas 3 dos 4 processos computam (75% da infraestrutura), o *Speedup* atinge tetos bem definidos: na Visão Completa, estabiliza-se em $1{,}44\times - 1{,}46\times$ (em 4K),

1,39 × −1,45× (em 8K) e 1,43 × −1,45× (em 16K). No Seahorse, onde a carga interna por linha é acentuada, a partição em meia-linha atinge até 1,72× em 16K, enquanto o modelo por linha inteira limita-se a 1,37×.

A estratégia de blocos contíguos (`block`) ilustra o colapso decorrente de ociosidade por barreira. Na Visão Completa, a concentração de iterações no centro horizontal faz com que os processos periféricos terminem em fração de segundo e permaneçam ociosos na primitiva coletiva `MPI_Gatherv`. Em $p = 2$, a simetria entre as metades superior e inferior ainda viabiliza $S_2 \approx 1{,}88\times$ até 1,96×. No entanto, com a subdivisão para $p = 3$ e $p = 4$, os núcleos extremos esvaziam-se rapidamente, colapsando a aceleração em $p = 4$ para patamares estagnados em torno de 0,97× a 0,98× (em 4K e 16K) e 0,92× (em 8K). Já no Seahorse, cuja heterogeneidade espacial distribui-se de modo distinto, o `block` sustenta ganhos superiores em $p = 4$, registrando 1,42× (em 4K), 1,61× (em 8K) e 1,68× (em 16K).

Por sua vez, a partição estática cíclica (`cyclic`) minimizou a fração de espera ao intercalar as linhas e manter todos os nós ativos. Beneficiando-se do uso dos núcleos físicos e do *Hyper-Threading*, superou as demais abordagens em todas as resoluções, escalando até $p = 3$ (atingindo 2,57 × −2,68×) e fechando em $p = 4$ com 2,00× (4K), 2,04× (8K) e 2,19× (16K) na Visão Completa, e com 1,52× (4K), 1,84× (8K) e 1,93× (16K) no Seahorse. Em contraste absoluto, o dinamismo com coordenador participativo (`dynamic_part`) sofreu degradação progressiva devido à contenção de mensagens do despachante, decaindo com o aumento de processos até registrar severa desaceleração em $p = 4$ ($S_4 \approx 0{,}36 \times -0{,}41\times$).

As Figuras 1, 2 e 3 apresentam o comportamento detalhado de escalabilidade forte empírica para as resoluções de 4K, 8K e 16K, confrontando as curvas experimentais com a linha de aceleração linear ideal e os limites físicos da máquina.

3.5.1 Lei de Gustafson: Escalabilidade Fraca e Diluição de Sobrecarga

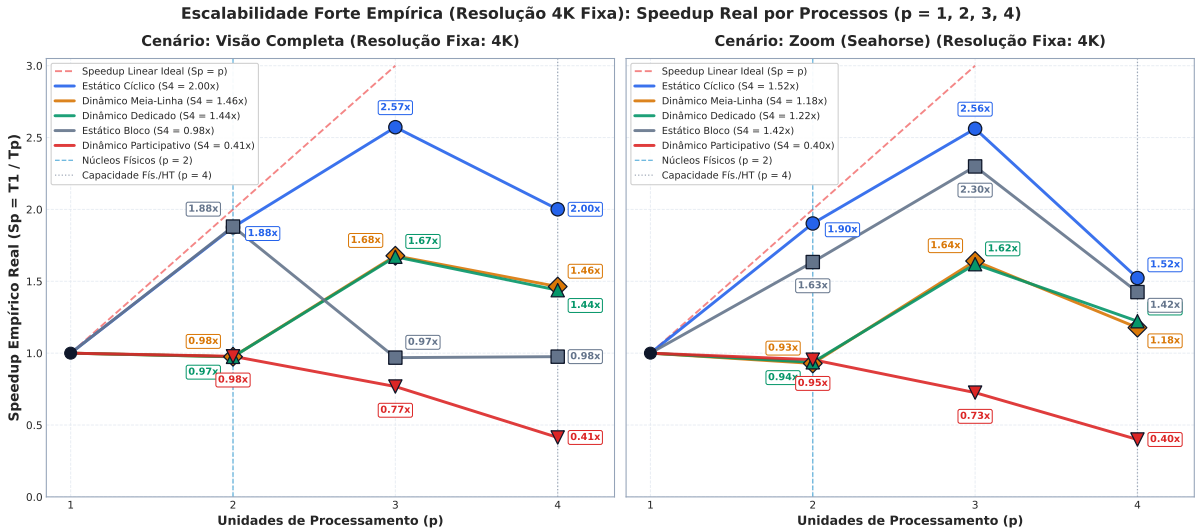
Formulada por John Gustafson (1988), a Lei de Gustafson-Barsis modela o comportamento de sistemas em regime de escalabilidade fraca, em que a carga de trabalho cresce proporcionalmente à quantidade de recursos de processamento alocados. No modelo teórico clássico, a aceleração escalada é expressa em função da fração sequencial s:

$$S_{\text{Gustafson}}(p) = s + p(1 - s) = p - s(p - 1) \tag{3.3}$$

Experimentalmente, mantendo a carga por processo constante ($W/p = \text{constante}$), a eficiência de escalabilidade fraca (E_{weak}) e o *Speedup* escalado empírico (S_p) calculados diretamente a partir dos tempos medidos são dados por:

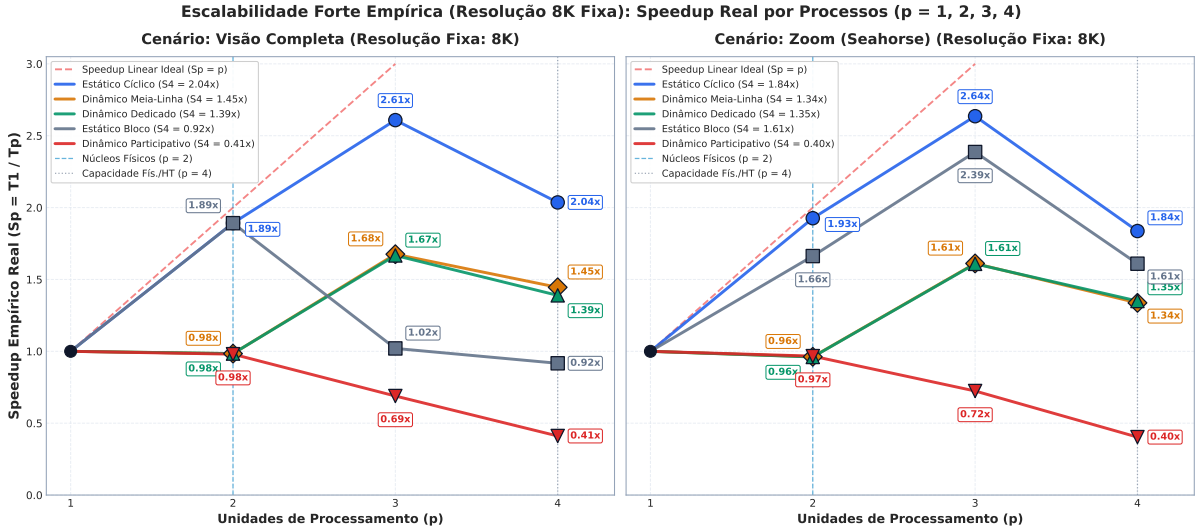
$$E_{\text{weak}}(p) = \frac{T_1(W)}{T_p(p \cdot W)} \times 100\% \tag{3.4}$$

Figura 1 – Curvas de escalabilidade forte empírica sob resolução 4K fixa (4096×4096 pixels) para $p \in \{1, 2, 3, 4\}$, confrontando os cenários de Visão Completa e Zoom no Seahorse.



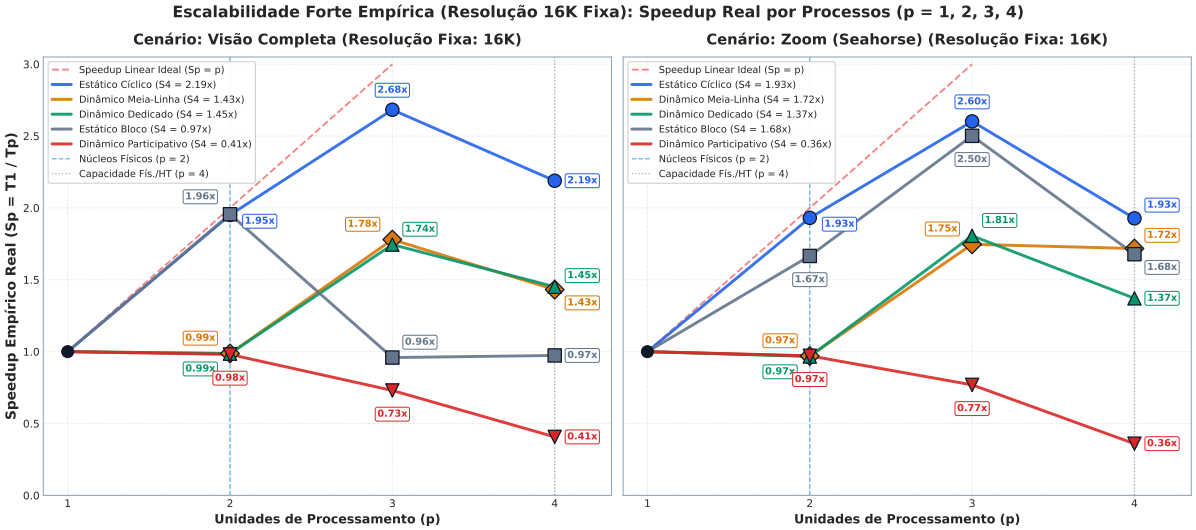
Fonte: Própria.

Figura 2 – Curvas de escalabilidade forte empírica sob resolução 8K fixa (8192×8192 pixels) para $p \in \{1, 2, 3, 4\}$, confrontando os cenários de Visão Completa e Zoom no Seahorse.



Fonte: Própria.

Figura 3 – Curvas de escalabilidade forte empírica sob resolução 16K fixa (16384 × 16384 pixels) para $p \in \{1, 2, 3, 4\}$, confrontando os cenários de Visão Completa e Zoom no Seahorse.



Fonte: Própria.

$$S_p = p \cdot \frac{E_{\text{weak}}}{100\%} = p \cdot \frac{T_1(W)}{T_p(p \cdot W)} \tag{3.5}$$

em que $T_1(W)$ é o tempo sequencial basal da matriz de referência e $T_p(p \cdot W)$ é o tempo paralelo obtido com p processos sobre a matriz de dimensão quadruplicada.

Nos experimentos realizados, a escalabilidade fraca foi avaliada em dois saltos proporcionais, mantendo a transição de $p = 1$ para $p = 4$ processos. Sendo assim, o Salto 1 expandiu a matriz de 4096×4096 (4K) para 8192×8192 (8K), fixando a carga de trabalho em 16,8 milhões de pixels por processo, enquanto o Salto 2 progrediu de 8192×8192 (8K) para 16384×16384 (16K), elevando a densidade por núcleo para 67,1 milhões de pixels. Os resultados obtidos em ambos os saltos são apresentados na Figura 4.

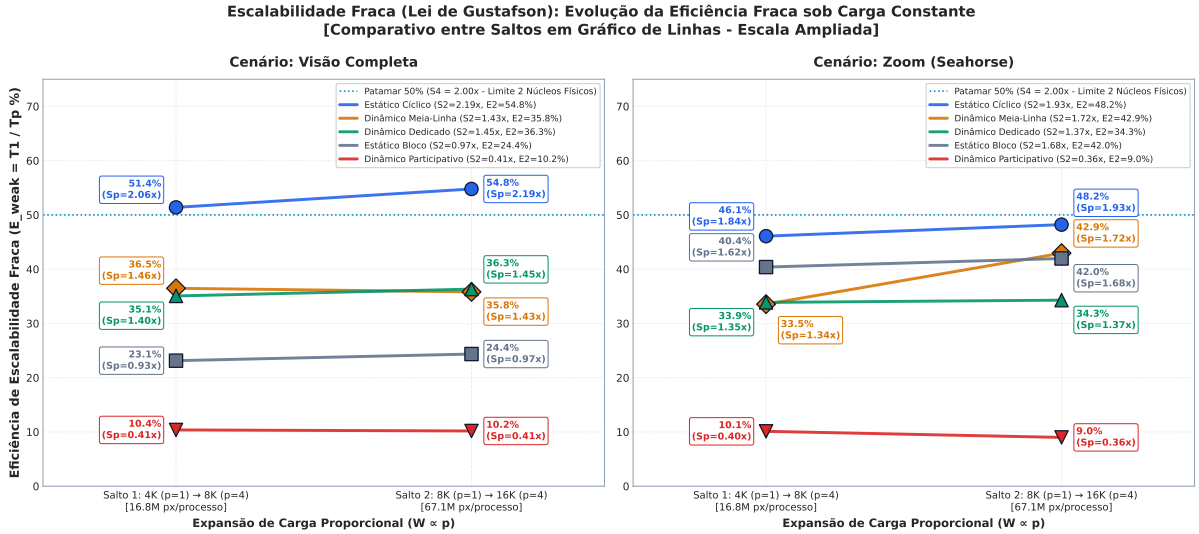
A distribuição estática cíclica (`cyclic`) comprovou com exatidão a hipótese de Gustafson ao registrar trajetórias ascendentes de eficiência em ambos os domínios geométricos, superando inclusive a linha teórica de 50% de eficiência fraca (equivalente ao limite nominal dos 2 núcleos físicos da CPU, onde $S_p = 2{,}00\times$). Na Visão Completa, a eficiência fraca evoluiu de 51,4% ($S_p = 2{,}06\times$) no primeiro salto para 54,8% ($S_p = 2{,}19\times$) no segundo salto. No Seahorse, o ganho acompanhou trajetória análoga, subindo de 46,1% ($S_p = 1{,}84\times$) para 48,2% ($S_p = 1{,}93\times$). Esse ganho sistemático ocorre porque o custo de coleta no `MPI_Gatherv` cresce em taxa sublinear em relação à massa de cálculo por linha, permitindo que as unidades funcionais via *Hyper-Threading* permaneçam ocupadas por períodos consideravelmente mais longos.

Nas abordagens sob demanda com coordenador dedicado, a diluição de sobrecarga evidenciou o contraste imposto pela topologia de divisão de tarefas. Na decomposição por

meia-linha (`half-row`) aplicada ao Seahorse, observou-se uma expressiva curva de ascensão: o rendimento saltou de 33,5% ($S_p = 1{,}34\times$) para 42,9% ($S_p = 1{,}72\times$), aproximando-se da distribuição cíclica ao mitigar o desbalanceamento local das espirais densas. Por outro lado, a abordagem dinâmica padrão (`dynamic`) estabilizou em platôs rígidos em ambos os cenários (em torno de 35,1% → 36,3% na Visão Completa e 33,9% → 34,3% no Seahorse), atestando que a improdutividade do processo coordenador (25% do hardware alocado sem computar pixels) impõe um freio estrutural que impede a conversão plena da escala em aceleração útil.

Por fim, o particionamento em bloco contíguo (`block`) refletiu a influência decisiva da distribuição espacial do fractal sobre a escalabilidade fraca. Enquanto no Seahorse a maior cobertura de estruturas complexas evitou a paralisação completa das faixas extremas, sustentando eficiências consistentes entre 40,4% e 42,0% ($S_p = 1{,}62\times \rightarrow 1{,}68\times$), na Visão Completa o confinamento da cardioide no centro horizontal condenou os processos periféricos à ociosidade precoce, mantendo o rendimento estagnado em patamares reduzidos de 23,1% a 24,4% ($S_p < 1{,}00\times$). Em contrapartida absoluta a todos os demais modelos, a variante com coordenador participativo (`dynamic_part`) estagnou entre 9,0% e 10,4% de eficiência fraca ($S_p \approx 0{,}36\times - 0{,}41\times$), provando que a sobrecarga de bloqueio e a contenção no despachante não são diluídas pela expansão do volume de dados.

Figura 4 – Evolução da eficiência de escalabilidade fraca ($E_{\text{weak}} = T_1/T_p \times 100\%$) e aceleração correspondente (S_p) sob expansão proporcional de carga ($W \propto p$), comparando os cenários de Visão Completa e Zoom no Seahorse entre o Salto 1 (16,8 Mpx/processo) e o Salto 2 (67,1 Mpx/processo).

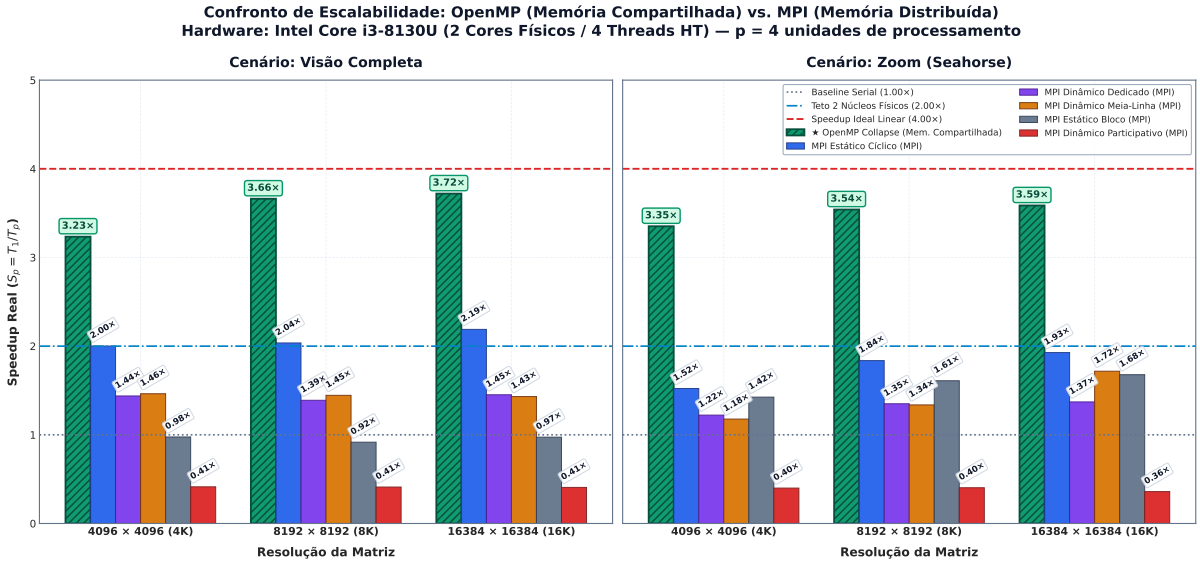


Fonte: Própria.

3.6 Comparativo entre Paradigmas: Memória Compartilhada (OpenMP) vs. Memória Distribuída (MPI)

A comparação direta entre a Etapa 1 (OpenMP) e a Etapa 2 (MPI) mostra como a forma de gerenciar a memória e organizar o trabalho afeta o desempenho em problemas com divisão de carga muito desigual. Em ambos os testes, utilizou-se o processador Intel Core i3-8130U, que conta com 2 núcleos físicos e 4 núcleos lógicos (*threads*) via *Hyper-Threading*, executando com carga total de $p = 4$ unidades de processamento. A Figura 5 resume o ganho de velocidade (*Speedup* real, $S_p = T_1/T_p$) entre a melhor versão em memória compartilhada (`OpenMP Collapse`) e as estratégias em memória distribuída com MPI, tanto na Visão Completa quanto no Zoom do Vale dos Cavalos-Marinhos (*Seahorse Valley*).

Figura 5 – Confronto de *Speedup* real (S_p) entre a melhor versão OpenMP e as abordagens MPI ($p = 4$).



A grande diferença de desempenho entre os dois modelos está em como os dados são acessados e como as tarefas são distribuídas. No OpenMP, a cláusula `collapse(2)` unida à política dinâmica (`schedule(dynamic)`) trabalha direto na memória RAM compartilhada. A entrega de novas partes da imagem para cada núcleo é controlada pelo próprio processador através de operações rápidas na memória *cache* L3. Como quase não se perde tempo organizando essa fila, o OpenMP aproveitou ao máximo os recursos do chip e alcançou acelerações quase ideais: 3,23× (em 4K), 3,66× (em 8K) e 3,72× (em 16K) na Visão Completa, além de manter índices entre 3,35× e 3,59× no Seahorse. Esses resultados superaram com facilidade o limite de 2 núcleos físicos (2,00×), comprovando que as tarefas simultâneas do *Hyper-Threading* conseguiram manter o processador ocupado o tempo todo.

Por outro lado, no MPI os processos não compartilham o mesmo espaço de memória.

Mesmo quando todos os processos rodam no mesmo computador (onde as trocas usam a memória interna da máquina), eles continuam completamente isolados uns dos outros. Isso significa que o programa precisa copiar dados entre áreas de memória, montar pacotes com identificadores (*tags*) e enviar mensagens explícitas de envio e recebimento, além de juntar os pedaços finais em uma operação de coleta coletiva com `MPI_Gatherv`.

Essa barreira atrapalhou bastante as tentativas de copiar no MPI o modelo dinâmico por demanda que funcionou tão bem no OpenMP. Na versão dinâmica tradicional com mestre dedicado, o processo 0 fica reservado exclusivamente para entregar linhas e receber os resultados prontos. Como o teste usou apenas 4 processos, deixar 1 processo só organizando o trabalho representou a perda de 25% de toda a capacidade de cálculo da máquina, restando apenas 3 processos para calcular os pixels. Assim, mesmo que os trabalhadores tenham terminado seus cálculos exatamente no mesmo instante, o *Speedup* ficou limitado a marcas entre 1,39× e 1,45× na Visão Completa e de 1,35× a 1,37× no Seahorse. A divisão dinâmica em meia-linha melhorou a divisão de tarefas nas regiões mais densas de zoom, chegando a 1,72× em 16K, mas ainda ficou bem atrás do OpenMP.

A tentativa de colocar o coordenador para calcular tarefas nas suas horas vagas, na versão participativa, resultou no pior desempenho entre todos os testes. Como o cálculo do fractal exige esforço contínuo e pesado de conta, o coordenador demorava muito para terminar suas próprias linhas e verificar as mensagens que chegavam com a função `MPI_Iprobe`. Essa demora fez com que os trabalhadores ficassem parados esperando novas tarefas (*worker starvation*), fazendo o programa rodar mais devagar do que a própria versão sequencial de referência, com acelerações muito baixas entre 0,36× e 0,41×.

A única abordagem em MPI que conseguiu encostar e superar a marca dos 2 núcleos físicos foi a estática cíclica. Ao intercalar as linhas da imagem de forma alternada (*round-robin*) antes de iniciar as contas, essa estratégia misturou trechos fáceis e difíceis por igual entre todos os processos, sem precisar trocar nenhuma mensagem durante a execução. Toda a comunicação ficou concentrada em um único envio ao final do cálculo, garantindo *Speedups* de 2,00× (4K), 2,04× (8K) e 2,19× (16K) na Visão Completa, e 1,93× em 16K no Seahorse.

Essa comparação comprova que, para resolver problemas com divisão de trabalho muito desigual em um único computador (*single-node*), a sincronização em memória compartilhada com OpenMP é muito superior, pois equilibra o trabalho de forma imediata sem desperdiçar núcleos de cálculo. O modelo MPI perde eficiência nesse cenário porque o envio constante de mensagens e a reserva de um processo coordenador custam caro quando se tem poucos núcleos. Dessa forma, para o MPI rodando em uma única máquina, a divisão intercalada (cíclica) mostrou ser a melhor opção, pois equilibra o esforço sem gastar tempo trocando mensagens durante o processamento.

4 CONCLUSÃO

Esta segunda etapa permitiu entender, na prática, os desafios de programar para computadores que usam memória distribuída com a biblioteca MPI. O cálculo do Conjunto de Mandelbrot deixou claro que, sem uma memória compartilhada entre os processos, a forma como dividimos a imagem e o custo de enviar mensagens são os fatores que realmente definem a velocidade do programa.

A divisão estática cíclica (`cyclic`) foi a mais rápida nos testes. Ao distribuir as linhas de forma alternada, ela espalhou as partes fáceis e difíceis da imagem por igual entre os processos, sem gastar tempo trocando mensagens no meio da conta. Na resolução maior de 16K, essa versão alcançou uma aceleração (*Speedup*) de 2,19× na Visão Global e 1,93× no Vale dos Cavalos-Marinhos, aproveitando mais de 95% da capacidade real dos dois núcleos físicos do processador através do *Hyper-Threading*. Já a divisão em blocos contínuos (`block`) sofreu uma queda drástica de desempenho na Visão Global ($F_{LB} = 97{,}5\%$), porque os processos que pegaram as bordas terminaram quase de imediato e ficaram parados esperando os processos centrais terminarem na barreira final.

Os modelos dinâmicos sob demanda mostraram a disputa entre equilibrar o trabalho e o custo da coordenação. Inicialmente, esses modelos foram propostos com o intuito de buscar uma abordagem de particionamento similar à que utilizamos com sucesso no OpenMP sob escalonamento dinâmico. A versão dinâmica tradicional (`dynamic`) e a versão dividida em meias-linhas (`half-row`) conseguiram dividir as tarefas de forma quase perfeita entre os trabalhadores ($F_{\text{LB}} \leq 0{,}1\%$). No entanto, elas esbarraram em um limite: como o processo 0 ficou dedicado apenas a entregar tarefas e não fez cálculos, 25% da máquina foi desperdiçada.

Por outro lado, a tentativa de colocar o processo coordenador para calcular na versão participativa (`dynamic_part`) causou um gargalo grave: o coordenador demorava muito calculando suas próprias linhas e não respondia rápido aos trabalhadores via `MPI_Iprobe`, deixando os outros processos parados sem tarefas e tornando o programa mais lento que a versão sequencial de referência ($S_p \approx 0{,}40\times$).

Esses resultados deixam claro que as diferenças fundamentais de arquitetura exigem soluções diferentes. Enquanto o modelo de memória compartilhada permite que a coordenação dinâmica opere com custo quase nulo diretamente na memória rápida, o paradigma de memória distribuída penaliza o dinamismo com a sobrecarga e a latência das mensagens. Isso evidencia que as estratégias de paralelização não podem ser simplesmente transpostas entre modelos, mas devem ser rigorosamente adaptadas à realidade e às restrições de comunicação de cada arquitetura.

Por fim, ao comparar com os resultados da etapa anterior, vimos que para rodar em um único computador a memória compartilhada (OpenMP) é mais eficiente, pois equilibra o trabalho direto na memória rápida sem precisar separar um processo só para gerenciar ($S_p \approx 3{,}72\times$). Já o MPI tem sua maior utilidade quando usamos vários computadores interligados em rede.

DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

Declaro que foram utilizadas ferramentas de inteligência artificial generativa, especificamente Gemini e Claude, como recursos de apoio durante a elaboração deste trabalho, referente à Etapa 1 da disciplina DEC107 - Processamento Paralelo. O seu emprego deu-se de forma restrita como suporte de produtividade no auxílio à depuração (*debugging*) de trechos de código e erros de sintaxe, e na revisão gramatical e textual do relatório. Ressalta-se que a especificação metodológica, o desenho dos algoritmos, os procedimentos de execução dos testes de desempenho e a validação crítica da corretude das matrizes de contagem foram integralmente conduzidos e supervisionados pelos autores.

REFERÊNCIAS

Mathigon. *Fractais: O Conjunto de Mandelbrot*. 2026. Acesso em: 24 ago. 2026. Disponível em: https://pt.mathigon.org/course/fractals/mandelbrot.

Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard, Version 4.0*. [S.l.], 2021. Acesso em: 25 out. 2026. Disponível em: https://www.mpi-forum.org/docs/mpi-4.0/mpi40-report.pdf.

ANEXO A — ENUNCIADO DO BENCHMARK DE MANDELBROT

Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time)

Disciplina DEC107 — Processamento Paralelo

Semestre 2026-2 — Agosto de 2026

---

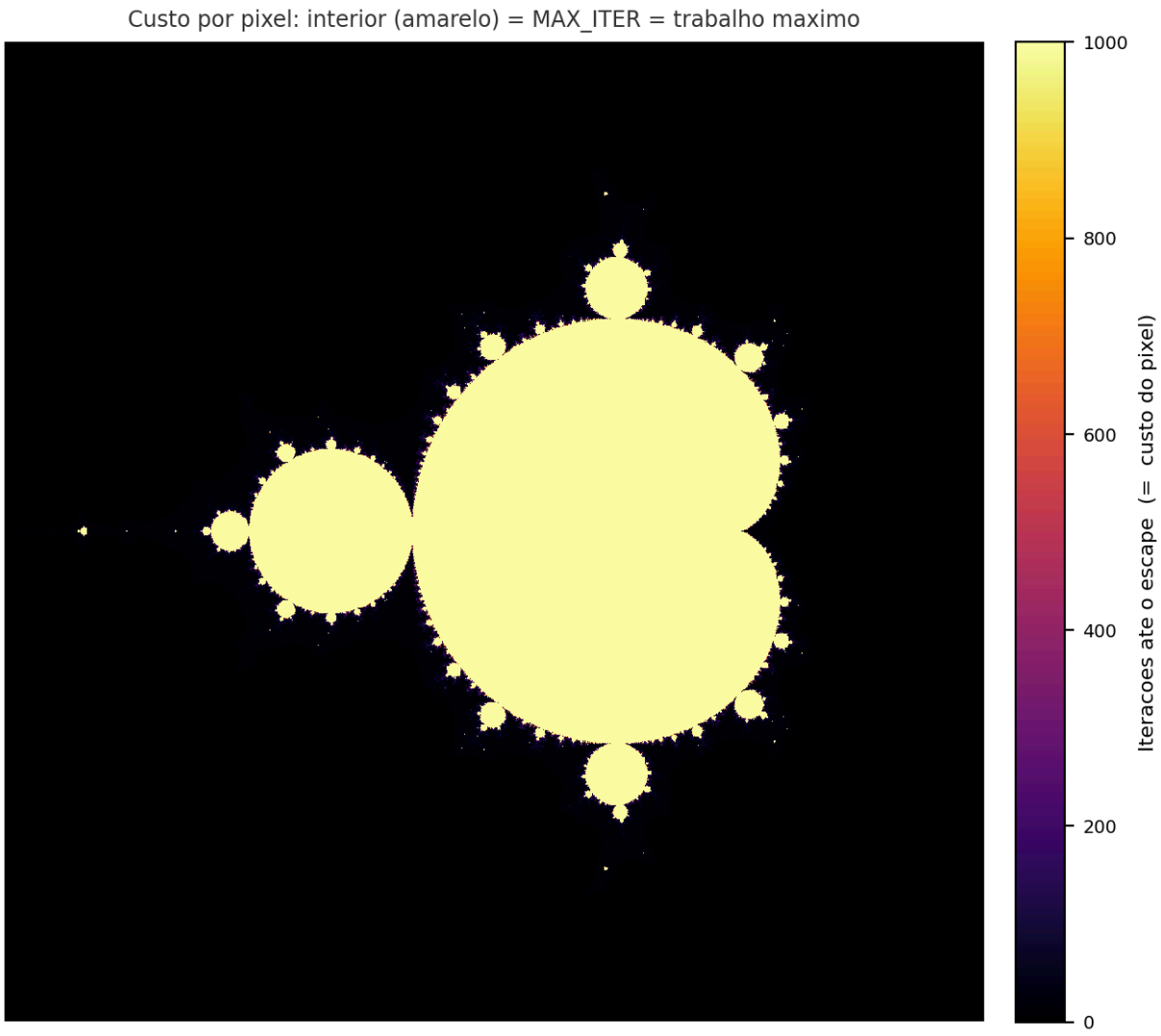
1. Contextualização e Motivação

Este projeto tem como objetivo principal avaliar o aprendizado dos conceitos de processamento paralelo da disciplina DEC107. Propomos a geração do conjunto de Mandelbrot pelo método de *escape-time* (tempo de escape) como problema central. Trata-se de um benchmark excelente por reunir três propriedades didáticas raras num mesmo problema: é embaraçosamente paralelo em sua estrutura básica, é fortemente **desbalanceado** em custo por elemento, e é determinístico — o que torna a validação de corretude simples e objetiva. A implementação de todas as etapas será realizada na linguagem C.

2. O que é o Conjunto de Mandelbrot?

O conjunto de Mandelbrot é o conjunto dos números complexos c para os quais a sequência definida por $z_0 = 0$ e $z_{n+1} = z_n^2 + c$ permanece limitada. Na prática, para cada ponto c do plano complexo, itera-se essa recorrência e conta-se quantas iterações são necessárias até que $|z|$ ultrapasse um raio de escape (tipicamente 2). Se o ponto não escapar dentro de um número máximo de iterações, considera-se que ele pertence ao conjunto.

Esse número de iterações até o escape — o *escape-time* — é usado para colorir cada pixel da imagem, produzindo a característica fronteira fractal do conjunto. O ponto central para este projeto é que **o custo computacional de cada pixel é proporcional ao seu escape-time**: pixels fora do conjunto tendem a escapar em poucas iterações, enquanto pixels dentro do conjunto (ou próximos da fronteira) consomem todas as iterações possíveis. Essa heterogeneidade de custo é a fonte natural de desbalanceamento de carga que o projeto explora.



3. Objetivo Geral

Desenvolver e otimizar um algoritmo de geração do conjunto de Mandelbrot por escape-time, aplicando técnicas de paralelismo em memória compartilhada (OpenMP), memória distribuída (MPI) e GPUs (CUDA), com ênfase na análise de balanceamento de carga e de escalabilidade.

4. Objetivos Específicos

Ao final do projeto, as duplas deverão ser capazes de:

1. Implementar uma versão serial e versões paralelas do gerador de Mandelbrot em C.
2. Aplicar e analisar o desempenho de paralelismo em OpenMP, MPI e CUDA.
3. Medir e interpretar métricas de desempenho como tempo de execução, Speedup e Eficiência.

4. Diagnosticar e mitigar o desbalanceamento de carga inerente ao problema, escolhendo estratégias de distribuição de trabalho apropriadas a cada paradigma.
5. Comparar o desempenho e a escalabilidade das diferentes abordagens paralelas, relacionando os resultados às leis de Amdahl e Gustafson.
6. Apresentar uma análise crítica dos resultados obtidos em relatórios e apresentações orais.

5. Descrição do Benchmark

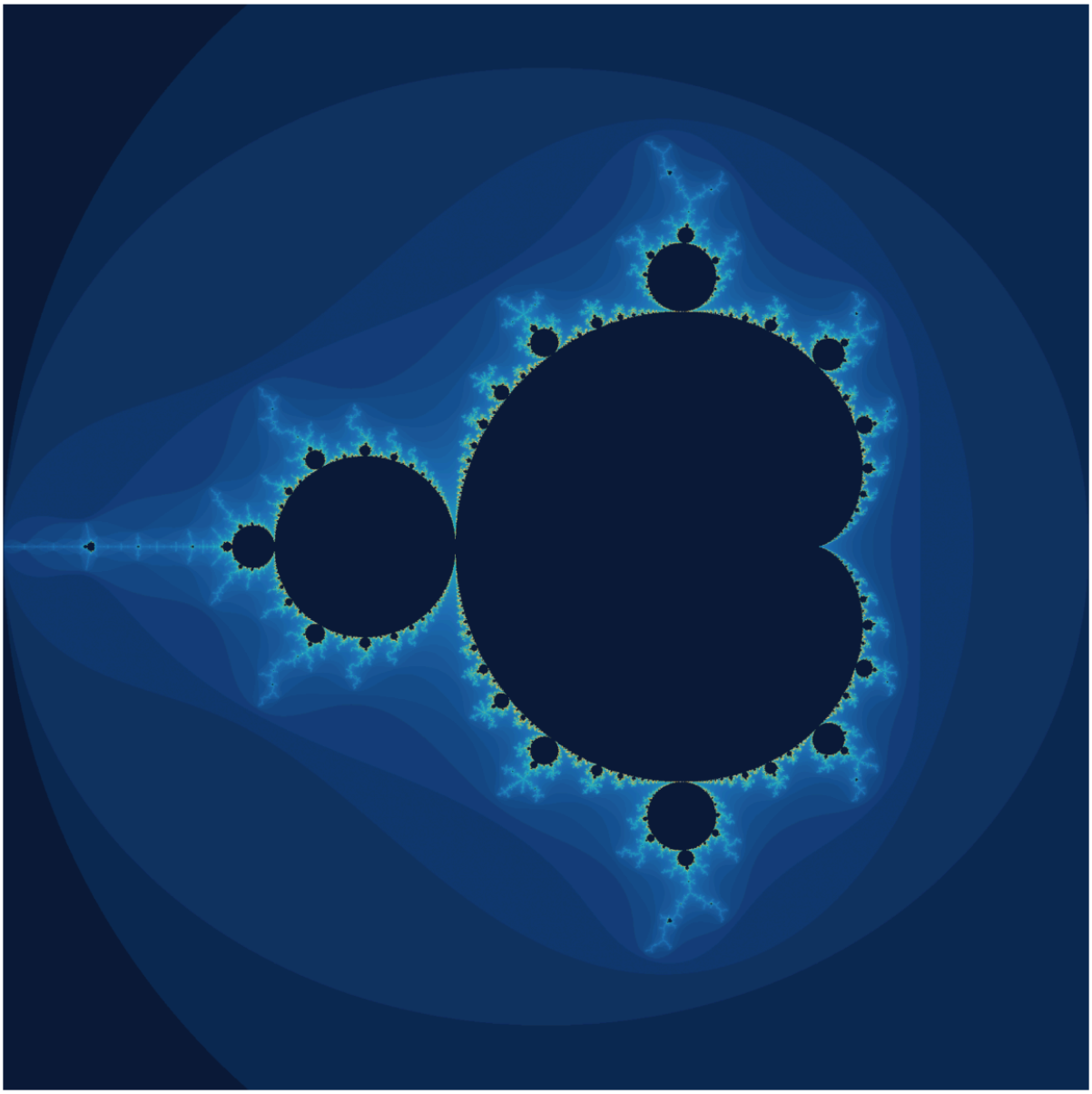
O benchmark consiste em calcular, para cada pixel de uma imagem que representa uma região do plano complexo, o escape-time da recorrência de Mandelbrot. A saída é uma matriz de contagens de iteração (uma por pixel), a partir da qual se gera a imagem colorida. Será fornecida uma versão serial de referência para validação da corretude.

5.1. Especificação do cálculo. Para o pixel de coordenadas de imagem (px, py), determina-se o ponto complexo c correspondente por mapeamento linear da região do plano definida abaixo. Itera-se $z_{n+1} = z_n^2 + c$ a partir de $z_0 = 0$, com critério de escape $|z|^2 > 4$ (equivalente a $|z| > 2$, evitando a raiz quadrada). A contagem de iterações até o escape, limitada por `MAX_ITER`, é o valor do pixel. Toda a aritmética deve usar ponto flutuante de dupla precisão (`double`).

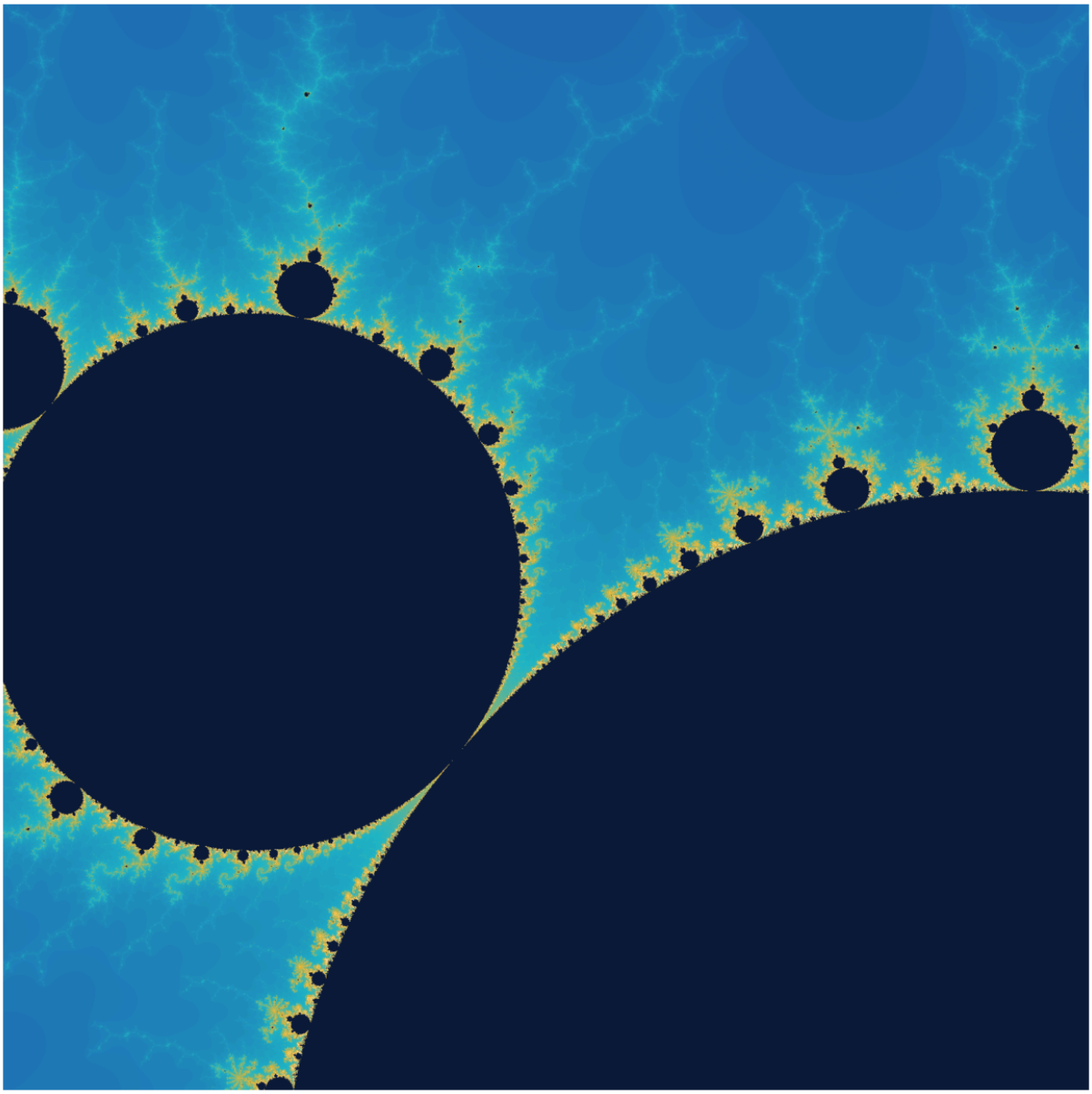
5.2. Input padrão (obrigatório). Para garantir a comparabilidade do ranking entre duplas, os seguintes parâmetros são fixos e devem ser usados em todos os experimentos oficiais:

- **Região do plano complexo (vista completa):** $\mathrm{Re} \in [-2{,}0, 1{,}0]$, $\mathrm{Im} \in [-1{,}5, 1{,}5]$.
- **Resolução base:** 4096 × 4096 pixels.
- **Número máximo de iterações:** `MAX_ITER = 1000`.
- **Raio de escape:** $|z| = 2$ (teste por $|z|^2 > 4$).
- **Precisão:** `double`.

Conjunto de Mandelbrot - vista completa (Re in [-2,1], Im in [-1.5,1.5])



Zoom: regiao de espirais (detalhe da fronteira fractal)

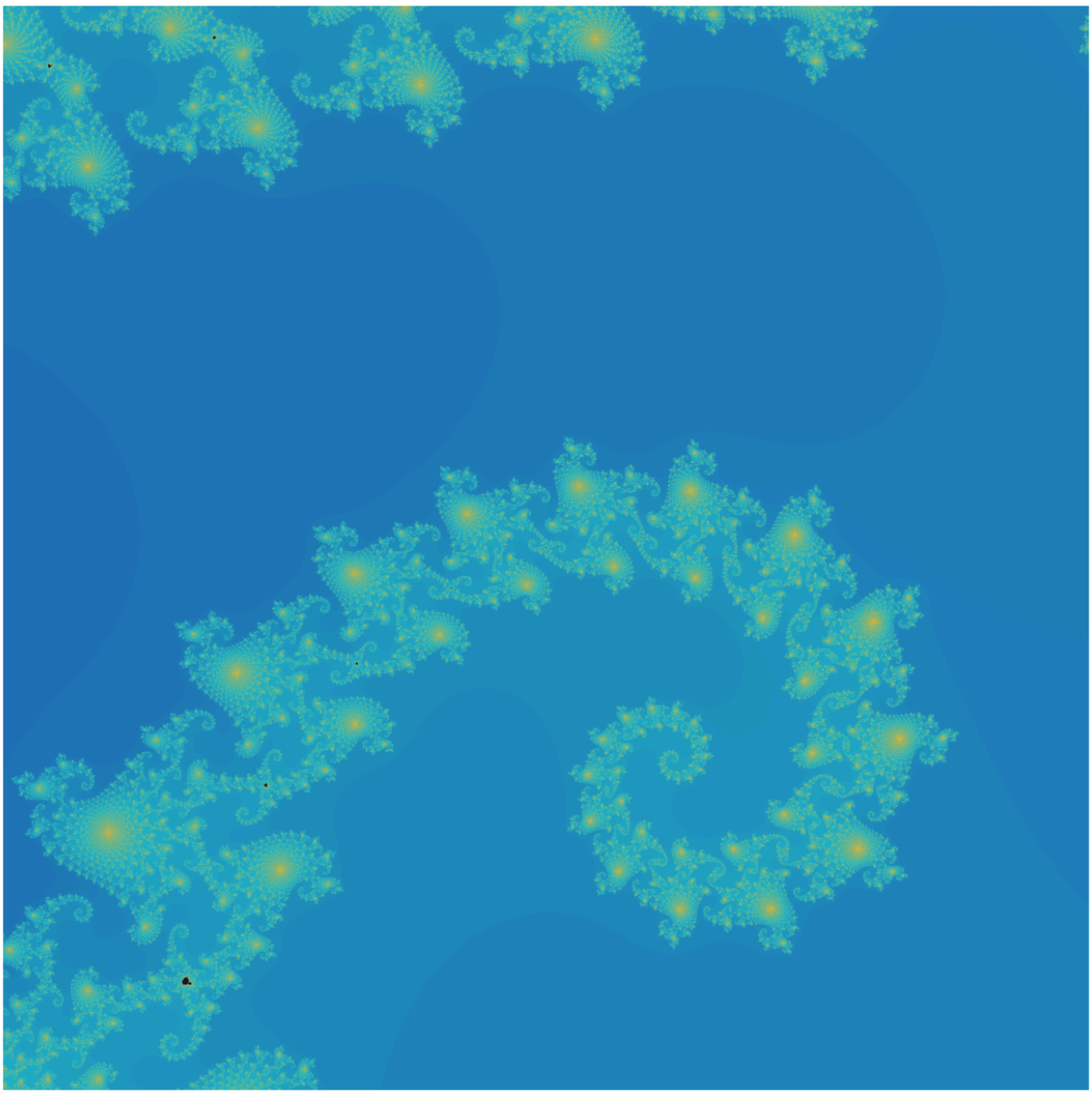


5.3. Casos de estresse (para as análises de escalabilidade). Além do caso base, os seguintes casos devem ser usados nas análises indicadas:

- **Escalabilidade forte (strong scaling):** manter a resolução base (4096 × 4096) e variar o número de threads/processos.
- **Escalabilidade fraca (weak scaling):** manter o trabalho por unidade de processamento aproximadamente constante, escalando a resolução (por exemplo 4096^2, 8192^2, 16384^2) proporcionalmente aos recursos.
- **Caso de desbalanceamento acentuado (obrigatório na análise de carga):** região de *zoom* no "vale dos cavalos-marinhos", centrada em (−0,743643887, 0,131825904) com

largura $3{,}0\times10^{-3}$ no eixo real (mantendo proporção da imagem), e `MAX_ITER = 5000`. Essa região concentra grande quantidade de pixels de custo máximo e evidencia problemas de distribuição de trabalho.

Zoom: vale dos cavalos-marinhos (centro -0.743643887 + 0.131825904i, MAX_ITER=5000)



5.4. Formato de saída. A saída canônica é a matriz de contagens de iteração gravada em arquivo binário (inteiros de 32 bits, ordem row-major), acompanhada de uma imagem em formato PGM ou PPM para inspeção visual. A imagem colorida serve à conferência humana; a validação de corretude é feita sobre a matriz de contagens.

5.5. Corretude. A saída paralela deve reproduzir a matriz de contagens da versão serial de referência. Por se tratar de um problema determinístico com contagens inteiras, a expectativa é de igualdade exata. Admite-se, no máximo, divergência em uma fração ínfima de pixels de fronteira (até 0,01% dos pixels, cada um diferindo em no máximo 1 iteração), desde que atribuível a reassociação de operações de ponto flutuante e explicitamente justificada no relatório. Divergências acima desse limite indicam erro de implementação e invalidam a pontuação de desempenho.

6. Regras do Projeto

1. O trabalho será realizado em duplas.
2. A submissão do código-fonte e dos relatórios será feita via GitHub.
3. Cada etapa do projeto exige a entrega de código-fonte, relatório técnico e resultados experimentais.
4. Os experimentos das etapas OpenMP e MPI devem ser realizados em computadores pessoais.
5. A etapa CUDA poderá exigir o uso de servidores remotos com hardware específico (GPU).
6. A corretude da implementação é obrigatória: a saída paralela deve ser equivalente à saída da versão serial de referência, conforme o critério da Seção 5.5.
7. O uso e a análise das métricas de desempenho são mandatórios em todas as etapas.
8. Os experimentos oficiais devem usar o input padrão da Seção 5.2; os casos de estresse da Seção 5.3 devem ser usados nas análises correspondentes.
9. Os relatórios devem registrar detalhadamente as condições do ambiente de execução (hardware, sistema operacional, compilador e parâmetros).

7. Cronograma das Três Etapas

7.1. Etapa 1 — OpenMP

Prazo: 2 semanas após a aula teórica.

Foco: Paralelismo em memória compartilhada. Distribuição do trabalho entre threads e, sobretudo, tratamento do desbalanceamento de carga por meio das políticas de escalonamento.

Entrega: Código serial e OpenMP, relatório da etapa.

Análise Principal no Relatório: Impacto do número de threads no desempenho; comparação entre as políticas de escalonamento (`static`, `dynamic` e `guided`) e o efeito do tamanho de *chunk*; Speedup, Eficiência e discussão fundamentada sobre balanceamento de carga, usando o caso de desbalanceamento acentuado da Seção 5.3.

7.2. Etapa 2 — MPI

Prazo: 2 semanas após a aula teórica.

Foco: Paralelismo em memória distribuída. Decomposição do domínio da imagem entre processos, estratégias de distribuição e custo de coleta do resultado.

Entrega: Código MPI, relatório da etapa.

Análise Principal no Relatório: Comparação entre pelo menos duas estratégias de distribuição — por exemplo, decomposição estática por blocos/faixas contíguas versus distribuição cíclica de linhas ou esquema mestre-trabalhador (*master-worker*) com balanceamento dinâmico; custo de comunicação (coleta da imagem via `Gather`/`Gatherv` e, quando aplicável, sobrecarga do mestre); Speedup, Eficiência, Fator de Balanceamento de Carga e comparação com a solução OpenMP.

7.3. Etapa 3 — CUDA

Prazo: 2 semanas após a aula teórica.

Foco: Paralelismo massivo em GPUs. Desenvolvimento do kernel de escape-time, configuração de blocos e grades de threads e uso eficiente da memória da GPU.

Entrega: Código CUDA, relatório final da etapa.

Análise Principal no Relatório: Desempenho do kernel; configuração de blocos/grade e sua influência no desempenho; discussão sobre **divergência de warps** decorrente do escape-time variável e eventuais estratégias de mitigação; custo de transferência host-device; Speedup, Eficiência e análise de ganhos e limitações em comparação com as versões anteriores.

8. Critérios de Avaliação

A avaliação será baseada nos seguintes critérios:

1. Corretude: A implementação deve produzir resultados corretos, equivalentes à versão serial (Seção 5.5).
2. Desempenho: A solução deve demonstrar ganhos significativos de tempo de execução em relação à versão serial.
3. Qualidade do Paralelismo: Uso adequado e otimizado das tecnologias (OpenMP, MPI, CUDA), incluindo o tratamento explícito do desbalanceamento de carga.
4. Análise dos Resultados: Capacidade da dupla de interpretar e discutir o comportamento da aplicação, os impactos do paralelismo e a escalabilidade (Amdahl/Gustafson).
5. Qualidade do Relatório e Apresentação Oral: Clareza, organização e correção técnica da documentação e da apresentação.

9. Métricas Obrigatórias

As seguintes métricas devem ser apresentadas e analisadas em todas as etapas:

- Tempo de Execução: O tempo total gasto para gerar a imagem (excluindo, quando pertinente, o tempo de escrita em disco, que deve ser reportado separadamente).
- Speedup: A razão entre o tempo de execução sequencial e o tempo de execução paralelo. Indica o ganho de velocidade.
- Eficiência: O Speedup dividido pelo número de processadores/threads. Indica o quão bem os recursos paralelos estão sendo utilizados.
- Fator de Balanceamento de Carga: Distribuição efetiva do trabalho entre threads/processos (por exemplo, razão entre o tempo do elemento mais carregado e o tempo médio, ou entre carga máxima e mínima). Esta métrica é central neste problema e deve ser analisada com o caso de desbalanceamento acentuado.
- Discussão sobre Escalabilidade: Análise de escalabilidade forte e fraca, relacionando os resultados às leis de Amdahl e Gustafson.

10. Sistema de Pontuação para Competição entre Duplas

Para incentivar a excelência e a análise crítica, será adotado um sistema de pontuação competitivo:

- A corretude da solução é um pré-requisito fundamental. Implementações incorretas não serão pontuadas em desempenho.
- Será estabelecido um ranking de desempenho para cada etapa, considerando o Speedup relativo obtido pela dupla em seu próprio hardware, sobre o input padrão da Seção 5.2. As duplas com os melhores desempenhos relativos receberão pontuação extra.
- Bônus serão concedidos por:
    - Análise técnica aprofundada e bem fundamentada nos relatórios, em especial no diagnóstico e na mitigação do desbalanceamento de carga.
    - Qualidade excepcional da apresentação oral.
    - Identificação e implementação de otimizações criativas (por exemplo, distribuição dinâmica de trabalho, redução de divergência em GPU, ou exploração de simetria do conjunto).

11. Orientações para os Relatórios

Cada relatório deve ser conciso e focar nos resultados da etapa. Deve incluir:

- Introdução: Breve descrição do objetivo da etapa.
- Metodologia: Detalhes da implementação paralela e da estratégia utilizada, incluindo a estratégia de distribuição de trabalho.
- Ambiente de Execução: Registro de hardware, sistema operacional, compilador e parâmetros de execução.

- Resultados: Apresentação das métricas obrigatórias (tempos, Speedup, Eficiência, balanceamento de carga) em gráficos claros.
- Análise: Discussão dos resultados, comparação com a versão serial (e com etapas anteriores, quando aplicável), identificação de gargalos e oportunidades de otimização, com atenção ao desbalanceamento e à escalabilidade.
- Conclusão: Resumo dos principais aprendizados e desafios da etapa.

12. Considerações Finais

Este projeto oferece uma oportunidade única para aplicar e aprofundar os conhecimentos em processamento paralelo em um contexto prático e visualmente rico. Ao contrário de um problema regularmente balanceado, a geração do conjunto de Mandelbrot expõe as duplas ao desafio central do HPC de distribuir trabalho heterogêneo de forma eficiente — tema que se manifesta de maneira distinta em cada um dos três paradigmas. A competição saudável entre as duplas visa estimular a busca por soluções eficientes e aprimorar a capacidade de análise crítica. O foco é no aprendizado e na compreensão dos desafios e benefícios do paralelismo em HPC.